

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/324783775>

Text to Image Synthesis Using Generative Adversarial Networks

Thesis · April 2018

DOI: 10.13140/RG.2.2.35817.39523

CITATIONS

23

READS

19,531

1 author:



Cristian Bodnar

University of Cambridge

21 PUBLICATIONS 199 CITATIONS

SEE PROFILE

Text to Image Synthesis Using Generative Adversarial Networks



Cristian Bodnar

Supervisor: Dr Jon Shapiro

University of Manchester
School of Computer Science

A final year report submitted for the degree of
BSc Computer Science

May, 2018

This report is dedicated to my brother Andrei

Acknowledgements

I would like to thank my supervisor Dr Jon Shapiro for his guidance and constructive suggestions during the planning and development of this research work. I also want to express my gratitude to Teodora Stoleru for her assistance, patience, and useful critique of this report. Finally, I wish to thank my family for their support and encouragement throughout my studies.

Abstract

Generating images from natural language is one of the primary applications of recent conditional generative models. Besides testing our ability to model conditional, highly dimensional distributions, text to image synthesis has many exciting and practical applications such as photo editing or computer-aided content creation. Recent progress has been made using Generative Adversarial Networks (GANs). This material starts with a gentle introduction to these topics and discusses the existent state of the art models. Moreover, I propose Wasserstein GAN-CLS, a new model for conditional image generation based on the Wasserstein distance which offers guarantees of stability. Then, I show how the novel loss function of Wasserstein GAN-CLS can be used in a Conditional Progressive Growing GAN. In combination with the proposed loss, the model boosts by 7.07% the best Inception Score (on the Caltech birds dataset) of the models which use only the sentence-level visual semantics. The only model which performs better than the Conditional Wasserstein Progressive growing GAN is the recently proposed AttnGAN which uses word-level visual semantics as well.

Contents

1	Background	1
1.1	Text to Image Synthesis	1
1.1.1	Datasets	2
1.2	Generative Models	3
1.3	Generative Adversarial Networks	3
1.3.1	Conditional Generative Adversarial Networks	6
1.3.2	Text Embeddings	7
2	State of the Art Models	9
2.1	Method	9
2.2	GAN-CLS (Conditional Latent Space)	9
2.2.1	Model Architecture	10
2.2.2	Adapting GAN Loss to Text-Image Matching	11
2.2.3	Training	12
2.2.4	Results	12
2.3	Stacked GANs	12
2.3.1	Text Embedding Augmentation	13
2.3.2	Model Architecture	14
2.3.3	Training	15
2.3.4	Results	15
2.4	Other Models	16
3	Research	18
3.1	Wasserstein GAN-CLS	18
3.1.1	Wasserstein GAN	19
3.1.2	Conditioning Wasserstein GAN	21
3.1.3	Architecture	22
3.1.4	Training	22
3.1.5	Results	23
3.2	Conditional Progressively Growing GANs	23

3.2.1	Architecture and Training	23
3.2.2	The Need for a Stable Loss Function	26
3.2.3	Results	27
4	Evaluation and Comparisons	30
4.1	The Inception Score	30
4.1.1	Evaluation of Text-Image Matching	31
4.2	Inception Score Results	32
4.3	Side by Side Comparison of the Models	33
5	Reflection and Conclusion	39
5.1	Planning and Management	39
5.2	Conclusion	40
A	Proof of Theorem 1.3.1	41
B	Notions of Deep Learning	43
B.1	Neural Networks	43
B.1.1	Backpropagation	43
B.1.2	Activation Functions	44
B.2	Normalisation Techniques	45
B.2.1	Batch Normalisation	45
B.2.2	Layer Normalisation	45
B.3	Convolutional Layers	46
B.4	Residual Layers	47
C	Additional Explanations of Wasserstein GANs	48
C.1	Manifolds	48
C.2	Lipschitz Functions	48
D	Samples Generated on the Birds Dataset	49
E	Conditional PGGANs	55
E.1	More on Architecture and Training	56
	Bibliography	57

List of Figures

1.1	Dataset samples	2
1.2	The GAN framework	4
1.3	Saddle point example	5
1.4	GANs as probabilistic graphical models	6
1.5	Char-CNN-RNN encoder	7
2.1	Architecture of Customised GAN-CLS	10
2.2	GAN-CLS Samples	11
2.3	GAN-CLS Interpolations	12
2.4	Augmentation in the conditional space	13
2.5	VAE reparametrisation trick	14
2.6	The architecture of StackGAN	15
2.7	StackGAN Stage I Samples	15
2.8	StackGAN Stage I interpolations	16
2.9	StackGAN Stage II Samples	16
2.10	StackGAN Stage II interpolations	17
2.11	StackGAN Stages	17
3.1	Wasserstein distance	19
3.2	WGAN-GP linear interpolations	20
3.3	Conditional Wasserstein GAN samples	22
3.4	Conditional Wasserstein GAN interpolations	23
3.5	Conditional PGGAN stages	24
3.6	PGGAN interpolation	25
3.7	Least Squares CPGGAN samples	26
3.8	Least Squares CPGGAN interpolations	27
3.9	Conditional Wasserstein PGGAN samples	27
3.10	Conditional Wasserstein PGGAN interpolations	28
3.11	Conditional Least Squares PGGAN stages	28
3.12	Conditional Wasserstein PGGAN stages	29
4.1	Inception architecture	30

4.2	Side by side comparison of 64×64 models (I)	33
4.3	Side by side comparison of 64×64 models (II)	34
4.4	Side by side comparison of 64×64 models (III)	35
4.5	Side by side comparison of 256×256 models (I)	35
4.6	Side by side comparison of 256×256 models (II)	36
4.7	Side by side comparison of 256×256 models (III)	36
4.8	Nearest neighbour analysis for 64×64 images	37
4.9	Nearest neighbour analysis for 256×256 images	38
B.1	Activation functions	44
B.2	Convolution	46
B.3	Transposed convolution	47
B.4	Residual layer	47
D.1	StackGAN Stage I birds dataset samples	49
D.2	StackGAN Stage I birds dataset interpolations	50
D.3	CWPGGAN birds dataset 64×64 samples	50
D.4	CWPGGAN birds dataset 64×64 interpolations	51
D.5	Comparison of 64×64 models on the birds dataset (I)	51
D.6	Comparison of 64×64 models on the birds dataset (II)	51
D.7	Comparison of 64×64 models on the birds dataset (III)	52
D.8	StackGAN Stage II birds dataset samples	52
D.9	StackGAN Stage II birds dataset interpolations	52
D.10	CWPGGAN birds dataset 256×256 samples	53
D.11	CWPGGAN birds dataset 256×256 interpolations	53
D.12	Comparison of 256×256 models on the birds dataset (I)	53
D.13	Comparison of 256×256 models on the birds dataset (II)	54

List of Tables

1.1	Summary statistics of the datasets.	2
4.1	Flowers dataset Inception Scores	32
4.2	Birds dataset Inception scores	32
5.1	First semester milestones	39
5.2	Second semester research directions	40
E.1	Conditional PGGAN architecture	55

List of Symbols

Numbers and Arrays

x	A scalar or element of a set $\mathcal{X}$ depending on the context
$\mathbf{X}$	A matrix
$\mathbf{x}$	A vector
$\mathbf{I}$	Identity matrix with dimensions inferred from context
X	A random variable
$\mathbf{X}$	A random vector
$\mathbf{x}^{(i)}$	The i^{th} example from a dataset
$\mathbf{x}_i$	The i^{th} element of the vector $\mathbf{x}$

Sets

$\mathcal{A}$	An arbitrary set
$\mathbb{R}$	The set of real numbers
$[a, b]$	The real interval which includes a and b
$(\mathcal{A}, d_{\mathcal{A}})$	A metric space on the set $\mathcal{A}$ with associated distance $d_{\mathcal{A}}$

Functions

$f : \mathcal{A} \rightarrow \mathcal{B}$	A function f with domain $\mathcal{A}$ and codomain $\mathcal{B}$
$f_{\boldsymbol{\theta}}(x)$	A function of x parametrised by $\boldsymbol{\theta}$
$G_{\boldsymbol{\theta}}$	The generator parametric function. Sometimes G is used instead.
$D_{\boldsymbol{\omega}}$	The discriminator parametric function. Sometimes D is used instead.
$\exp(x)$	Natural exponential function
$\log(x)$	Natural logarithm of x
$\ \mathbf{x}\ $	The L^2 norm of a vector $\mathbf{x}$

$\phi(t)$	The parametric function which takes a text as input and outputs a text embedding. The parameters are not explicitly specified to save space.
L_G	The loss function of the generator
L_D	The loss function of the discriminator
$IS(G)$	The Inception Score for a generator G
L_{GP}	Gradient penalty term
L_{LP}	Lipschitz penalty term
$F[x, f, f']$	A functional F which depends on $f(x)$ and $f'(x)$. $F[x, f]$ is used if F does not depend on f'
$\ f\ _L \leq K$	A K -Lipschitz function f for some constant K

Calculus

$\frac{df}{dx}$	The derivative of the function f with respect to x
$\frac{\partial f}{\partial x}$	The partial derivative of the multivariate function f with respect to x
f'	The derivative of the function f with respect to its argument
$\nabla_{\mathbf{x}} f(\mathbf{x})$	The gradient of the function f with respect to $\mathbf{x}$
$\int_{\mathcal{X}} f(x) dx$	The definite integral with respect to x over the set $\mathcal{X}$

Distributions and Information Theory

$P_X(x)$	A probability mass function of a discrete random variable X
$p_X(x)$	A probability density of a continuous random variable X
$\mathbb{P}_r$	The unknown data generating distribution of a dataset
$\mathbb{P}_g$	The distribution of a generative model
$\mathbb{P}_{ge}$	The joint distribution of the generated images and the text embeddings they were generated from
$\mathbb{P}_\eta$	The joint distributions of interpolations between real and generated images and text descriptions which match the real images used in the interpolation.
$\mathbb{P}_{r-mat}$	The distribution of real image and text pairs which match
$\mathbb{P}_{r-mis}$	The distribution of real image and text pairs which mismatch
$X \sim P$	A random variable X with distribution P

$\mathbb{E}_{X \sim P}[f(x)]$	The expected value of function f with respect to P
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	Multivariate normal distribution with mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$
$KL(P \parallel Q)$	The Kullback-Leibler divergence between P and Q
$JS(P \parallel Q)$	The Jensen-Shannon divergence between P and Q
$W(P, Q)$	The Wasserstein distance between P and Q
$U(a, b)$	Uniform distribution on the interval $[a, b]$
$U(a, b)^n$	Uniform distribution on the set $[a, b]^n$

Chapter 1

Background

This chapter contains a short introduction to the problem of text to image synthesis, generative models and Generative Adversarial Networks. An overview of these subjects is needed for the reader to understand the more complex ideas introduced in the later chapters.

1.1 Text to Image Synthesis

One of the most common and challenging problems in Natural Language Processing and Computer Vision is that of image captioning: given an image, a text description of the image must be produced. Text to image synthesis is the reverse problem: given a text description, an image which matches that description must be generated.

From a high-level perspective, these problems are not different from language translation problems. In the same way similar semantics can be encoded in two different languages, images and text are two different “languages” to encode related information.

Nevertheless, these problems are entirely different because text-image or image-text conversions are highly multimodal problems. If one tries to translate a simple sentence such as “This is a beautiful red flower” to French, then there are not many sentences which could be valid translations. If one tries to produce a mental image of this description, there is a large number of possible images which would match this description. Though this multimodal behaviour is also present in image captioning problems, there the problem is made easier by the fact that language is mostly sequential. This structure is exploited by conditioning the generation of new words on the previous (already generated) words. Because of this, text to image synthesis is a harder problem than image captioning.

The generation of images from natural language has many possible applications in the future once the technology is ready for commercial applications. People could create customised furniture for their home by merely describing it to a computer instead of spending many hours searching for the desired design. Content creators could produce content in tighter collaboration with a machine using natural language.

1.1.1 Datasets

The publicly available datasets used in this report are the Oxford-102 flowers dataset [25] and the Caltech CUB-200 birds dataset [35]. These two datasets are the ones which are usually used for research on text to image synthesis. Oxford-102 contains 8,192 images from 102 categories of flowers. The CUB-200 dataset includes 11,788 pictures of 200 types of birds. These datasets include only photos, but no descriptions. Nevertheless, I used the publicly available captions collected by Reed et al.[28] for these datasets using Amazon Mechanical Turk. Each of the images has five descriptions. They are at least ten words in length, they do not describe the background, and they do not mention the species of the flower or bird (Figure 1.1).



This pink flower has overlapping petals and yellow florets growing in the middle.



This bird is white with blue on its back and has a long, pointy beak.

Figure 1.1: A sample from the Oxford-102 dataset (left) and CUB-200 dataset (right), together with one of their associated descriptions collected by Reed et al. [28]

Because the number of images is small, I perform data augmentation. I apply random cropping and random left-right flipping of the images. I split the datasets into train and test datasets such that they contain disjoint classes of images. The datasets are summarised in Table 1.1.

The report is focused on the flowers dataset for practical reasons detailed in Chapter 5 where I discuss this decision. Nevertheless, a small number of experiments were run on the birds dataset as well.

Dataset/Number of images	Train	Test	Total
Flowers	7,034	1,155	8,192
Augmented flowers (256x256)	675,264	110,880	786,432
Birds	8,855	2,933	11,788
Augmented birds (256x256)	850,080	281,568	1,131,648

Table 1.1: Summary statistics of the datasets.

1.2 Generative Models

The task of text to image synthesis perfectly fits the description of the problem generative models attempt to solve. The current best text to image results are obtained by Generative Adversarial Networks (GANs), a particular type of generative model. Before introducing GANs, generative models are briefly explained in the next few paragraphs.

Before defining them, I will introduce the necessary notation. Consider a dataset $\mathbb{X} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ composed of m samples where $\mathbf{x}^{(i)}$ is a vector. In the particular case of this report, $\mathbf{x}^{(i)}$ is an image encoded as a vector of pixel values. The dataset is produced by sampling the images from an unknown data generating distribution $\mathbb{P}_r$, where r stands for real. One could think of the data generating distribution as the hidden distribution of the Universe which describes a particular phenomenon. A generative model is a model which learns to generate samples from a distribution $\mathbb{P}_g$ which estimates $\mathbb{P}_r$. The model distribution, $\mathbb{P}_g$, is a hypothesis about the true data distribution $\mathbb{P}_r$.

Most generative models explicitly learn a distribution $\mathbb{P}_g$ by maximising the expected log-likelihood $\mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r} \log(\mathbb{P}_g(\mathbf{x}|\boldsymbol{\theta}))$ with respect to $\boldsymbol{\theta}$, the parameters of the model. Intuitively, maximum likelihood learning is equivalent to putting more probability mass around the regions of $\mathcal{X}$ with more examples from $\mathbb{X}$ and less around the regions with fewer examples. It can be shown that the log-likelihood maximisation is equivalent to minimising the Kullback-Leibler divergence $KL(\mathbb{P}_r \parallel \mathbb{P}_g) = \int_{\mathcal{X}} \mathbb{P}_r \log \frac{\mathbb{P}_r}{\mathbb{P}_g} d\mathbf{x}$ assuming $\mathbb{P}_r$ and $\mathbb{P}_g$ are densities. One of the valuable properties of this approach is that no knowledge of the unknown $\mathbb{P}_r$ is needed because the expectation can be approximated with enough samples according to the weak law of large numbers.

Generative Adversarial Networks (GANs) [9] are another type of generative model which takes a different approach based on game theory. The way they work and how they compare to other models is explained in the next section.

1.3 Generative Adversarial Networks

Generative Adversarial Networks (GANs) solve most of the shortcomings of the already existent generative models:

- The quality of the images generated by GANs is better than the one of the other models.
- GANs do not need to learn an explicit density $\mathbb{P}_g$ which for complex distributions might not even exist as it will later be seen.
- GANs can generate samples efficiently and in parallel. For example, they can generate an image in parallel rather than pixel by pixel.
- GANs are flexible, both regarding the loss functions and the topology of the network which generates samples.

- When GANs converge, $\mathbb{P}_g = \mathbb{P}_r$. This equality does not hold for other types of models which contain a biased estimator in the loss they optimise.

Nevertheless, these improvements come at the expense of two new significant problems: the instability during training and the lack of any indication of convergence. GAN training is relatively stable on specific architectures and for carefully chosen hyper-parameters, but this is far from ideal. Progress has been made to address these critical issues which I will discuss in Chapter 3.

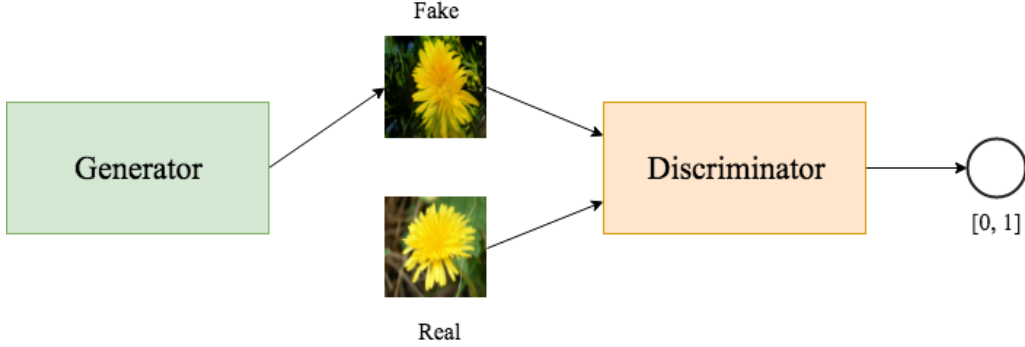


Figure 1.2: High-level view of the GAN framework. The generator produces synthetic images. The discriminator takes images as input and outputs the probability it assigns to the image of being real. A common analogy is that of an art forger (the generator) which tries to forge paintings and an art investigator (the discriminator) which tries to detect imitations.

The GAN framework is based on a game played by two entities: the discriminator (also called the critic) and the generator. Informally, the game can be described as follows. The generator produces images and tries to trick the discriminator that the generated images are real. The discriminator, given an image, seeks to determine if the image is real or synthetic. The intuition is that by continuously playing this game, both players will get better which means that the generator will learn to generate realistic images (Figure 1.2).

I will now show how this intuition can be modelled mathematically. Let $\mathbb{X}$ be a dataset of samples $\mathbf{x}^{(i)}$ belonging to a compact metric set $\mathcal{X}$ such as the space of images $[-1, 1]^n$. The discriminator learns a parametric function $D_{\omega} : \mathcal{X} \rightarrow [0, 1]$ which takes as input an image $\mathbf{x}$ and outputs the probability it assigns to the image of being real. Let $\mathcal{Z}$ be the range of a random vector $\mathbf{Z}$ with a simple and fixed distribution such as $p_{\mathbf{Z}} = \mathcal{N}(\mathbf{0}, \mathbf{I})$. The generator learns a parametric function $G_{\theta} : \mathcal{Z} \rightarrow \mathcal{X}$ which maps the states of the random vector $\mathbf{Z}$ to the states of a random vector $\mathbf{X}$. The states of $\mathbf{X} \sim \mathbb{P}_g$ correspond to the images the generator creates. Thus, the generator learns to map a vector of noise to images.

The easiest way to define and analyse the game is as a zero-sum game where D_{ω} and G_{θ} are the strategies of the two players. Such a game can be described by a value function $V(D, G)$ which in this case represents the payoff of the discriminator. The discriminator wants to maximise V while the generator wishes to minimise it. The payoff described by V must be proportional to the ability of D to distinguish between real and fake samples.

The value function from equation 1.1 which was originally proposed in [9] comes as a natural choice.

$$V(D, G) = \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_r}[\log(D(\mathbf{x}))] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}}[\log(1 - D(G(\mathbf{z})))] \quad (1.1)$$

On the one hand, note that $V(D, G)$ becomes higher when the discriminator can distinguish between real and fake samples. On the other hand, V becomes lower when the generator is performing well, and the critic cannot distinguish well between real and fake samples.

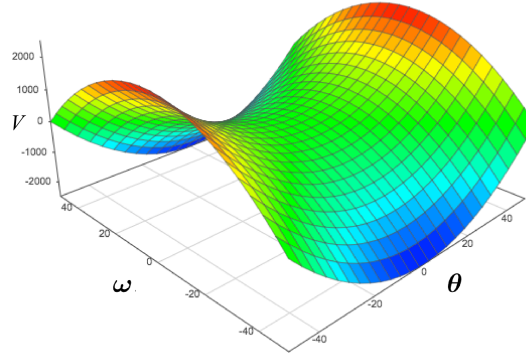


Figure 1.3: The Nash Equilibrium corresponds to a saddle point in space where V is at a minimum with respect to θ and at a maximum with respect to ω . Neither of the networks has any interest to change their parameters.

The difference from maximum likelihood models is that samples generated by the generator have an implicit distribution $\mathbb{P}_g$ determined by the particular value of θ . $\mathbb{P}_g$ cannot be explicitly evaluated. The discriminator forces the generator to bring $\mathbb{P}_g$ close to $\mathbb{P}_r$.

Zero-sum games are minimax games so the optimal parameters of the generator can be described as in Equation 1.2.

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \max_{\omega} V(D, G) \quad (1.2)$$

The solution of the minimax optimisation is a Nash Equilibrium (Figure 1.3). Theorem 1.3.1 describes the exciting properties of the equilibrium for non-parametric functions. The theorem confirms the intuition that as the two players play this game, the generator will improve at producing realistic samples.

Theorem 1.3.1. *The Nash equilibrium of the (non-parametric) GAN game occurs when:*

1. *The strategy of the discriminator is $D = \frac{\mathbb{P}_r}{\mathbb{P}_r + \mathbb{P}_g}$*
2. *The strategy G of the generator makes $\mathbb{P}_g = \mathbb{P}_r$.*

For completeness, I offer a slightly different and more detailed proof than the one from [9] in appendix A. I encourage the reader to go through it. Note that when the equilibrium is reached, $D = \frac{1}{2}$. The interpretation of Theorem 1.3.1 is that, at the end of the learning process, the generator learned $\mathbb{P}_r$ and the discriminator is not able to distinguish between real and synthetic data, so its best strategy is to output $\frac{1}{2}$ for any input it receives. At equilibrium, the discriminator has a payoff of $-\log(4)$ and the generator $\log(4)$, respectively.

In practice, the generator does not minimise $V(D, G)$. This would be equivalent to minimising $\mathbb{E}_{\mathbf{Z} \sim p_{\mathbf{Z}}}[\log(1 - D_{\omega}(G_{\theta}(\mathbf{z})))]$, but the function $\log(1 - D_{\omega}(G_{\theta}(\mathbf{z})))$ has saturating gradient when the discriminator is performing well and the function approaches $\log(1)$. This makes it difficult for the generator to improve when it is not performing well. Instead, the generator is minimising L_G from 1.3 whose gradient saturates only when the generator is already performing well. The loss L_D of the critic is also included in 1.3.

$$\begin{aligned} L_G &= -\mathbb{E}_{\mathbf{Z} \sim \mathbb{P}_{\mathbf{Z}}}[\log(D_{\omega}(G_{\theta}(\mathbf{z}))) \\ L_D &= -\mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r}[\log(D_{\omega}(\mathbf{x}))] - \mathbb{E}_{\mathbf{Z} \sim p_{\mathbf{Z}}}[\log(1 - D_{\omega}(G_{\theta}(\mathbf{z})))] \end{aligned} \quad (1.3)$$

With this generator loss function, the game is no longer a zero-sum game. Nevertheless, this loss works better in practice.

Based on the min-max expression 1.2, one might expect that the discriminator is trained until optimality for a fixed generator, the generator is trained afterwards, and the process repeats. This approach does not work for reasons which will become clear in Section 3.1.1. In practice, the networks are trained alternatively, for only one step each.

1.3.1 Conditional Generative Adversarial Networks

The original GAN paper [9] describes how one can trivially turn GANs into a conditional generative model. To generate data conditioned on some condition vector $\mathbf{c}$, $\mathbf{c}$ is appended to both the generator and the discriminator in any of their layers. The networks will learn to adapt and adjust their parameters to these additional inputs.

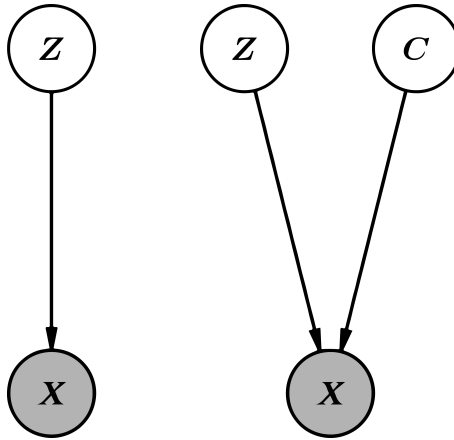


Figure 1.4: Probabilistic graphical model view of regular GANs (left) and conditional GANs (right).

Conditional GANs can also be seen from the perspective of a probabilistic graphical model (Figure 1.4). In the case of regular GANs, the noise $\mathbf{Z}$ influences the observable $\mathbf{X}$. For conditional *GANs*, both $\mathbf{Z}$ and $\mathbf{C}$ influence $\mathbf{X}$. In the particular case of text to image synthesis, the states $\mathbf{c}$ of $\mathbf{C}$ are vectors encoding a text description. How this encoding of a sentence into a vector is computed can vary, and it is discussed in Section 1.1.

1.3.2 Text Embeddings

The text descriptions must be vectorised before they can be used in any model. These vectorisations are commonly referred to as text embeddings. Text embedding models were not the focus of this work, and that is why the already computed vectorisations by Reed et al. [28] are used. Other state of the art models [29, 37] use the same embeddings and their usage makes comparisons between models easier.

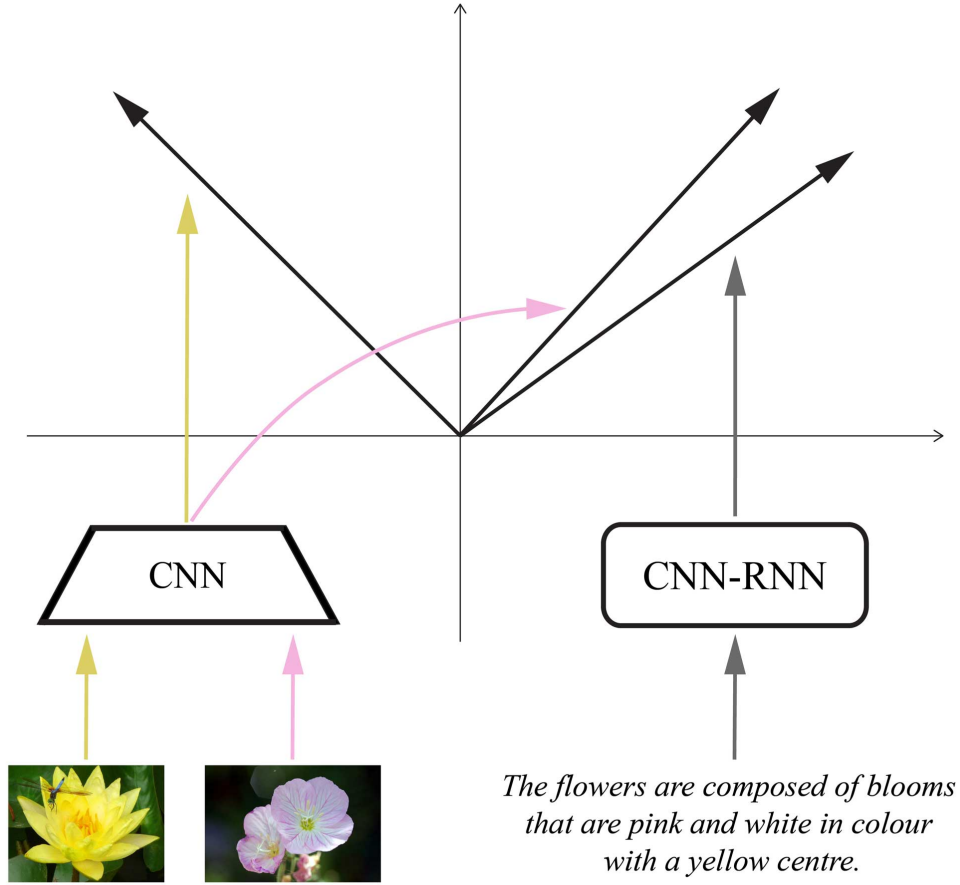


Figure 1.5: The char-CNN-RNN encoder maps images to a common embedding space. Images and descriptions which match are closer to each other. Here the embedding space is $\mathbb{R}^2$ to make visualisation easier. In practice, the preprocessed descriptions are in $\mathbb{R}^{1024}$.

The text embeddings are computed using the char-CNN-RNN encoder proposed in [28]. The encoder maps the images and the captions to a common embedding space such that images and descriptions which match are mapped to vectors with a high inner product.

For this mapping, a Convolutional Neural Network (CNN) processes the images, and a hybrid Convolutional-Recurrent Neural Network (RNN) transforms the text descriptions (Figure 1.5).

A common alternative is Skip-Thought Vectors [20] which is a pure language-based model. The model maps sentences with similar syntax and semantics to similar vectors. Nevertheless, the char-CNN-RNN encoder is better suited for vision tasks as it uses the corresponding images of the descriptions as well. The embeddings are similar to the convolutional features of the images they correspond to, which makes them visually discriminative. This property reflects in a better performance when the embeddings are employed inside convolutional networks.

Chapter 2

State of the Art Models

In this chapter, I first discuss in Section 2.1 the technical details of my implementation. Then, in Sections 2.2 and 2.3 I present in depth the state of the art models introduced by September 2017. Section 2.4 briefly mentions other state of the art models which have been proposed since I started this project.

2.1 Method

All the images included in this report generated by the described models are produced by my implementation of these models. All the models which are discussed are implemented in python 3.6 using the GPU version of TensorFlow [1].

TensorFlow is an open-source library developed by researchers from Google Brain and designed for high performance numerical and scientific computations. It is one of the most widely used libraries for machine learning research. TensorFlow offers both low level and high-level APIs which make development flexible and allow fast iteration. Moreover, TensorFlow makes use of the capabilities of modern GPUs for parallel computations to execute operations on tensors efficiently.

I run all the experiments on a Nvidia 1080Ti which I acquired for the scope of this project.

2.2 GAN-CLS (Conditional Latent Space)

Reed et al. [29] were the first to propose a solution with promising results for the problem of text to image synthesis. The problem can be divided into two main subproblems: finding a visually discriminative representation for the text descriptions and using this representation to generate realistic images.

In Section 1.3.2 I briefly described how good representations for the text descriptions could be computed using the char-CNN-RNN encoder proposed in [28]. In section 1.3.1 I also explained how Conditional GANs could be used to generate images conditioned on some vector c . GAN-CLS puts these two ideas together.

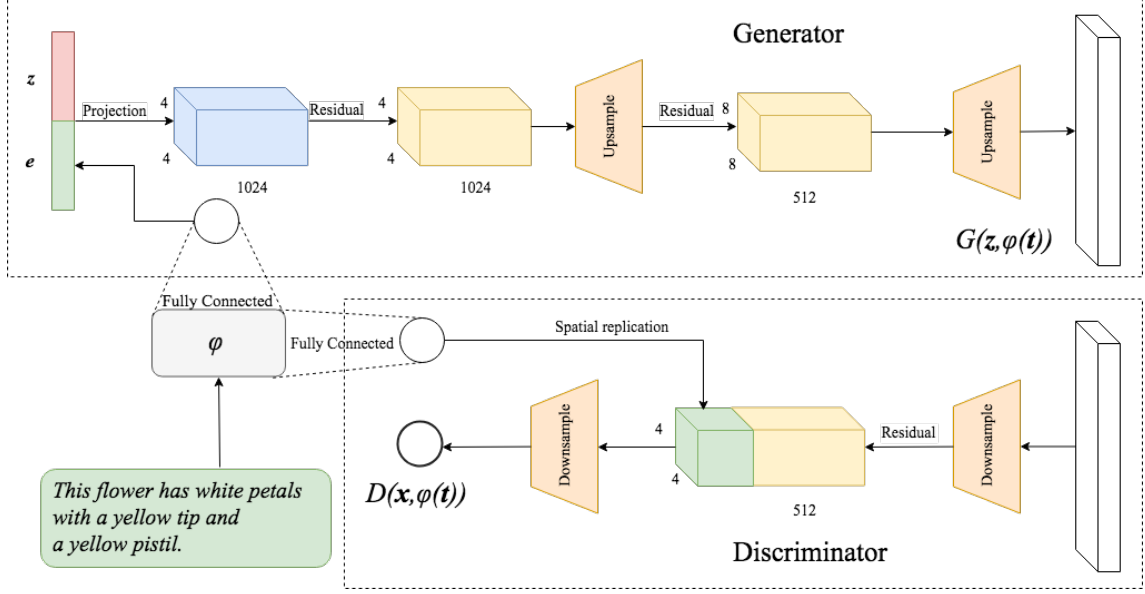


Figure 2.1: Architecture of the customised GAN-CLS. Two fully connected layers compress the text embedding $\phi(t)$ and append it both in the generator and the discriminator. In the discriminator, the compressed embeddings are spatially replicated (duplicated) before being appended in depth.

The functions $G(z)$ and $D(x)$ encountered in regular GANs become in the context of conditional GANs, $G(z, \phi(t))$ and $D(x, \phi(t))$, where $\phi: \Sigma^* \rightarrow \mathbb{R}^{N_\phi}$ is the char-CNN-RNN encoder, Σ is the alphabet of the text descriptions, t is a text description treated as a vector of characters and N_ϕ is the number of dimensions of the embedding. The text embedding $\phi(t)$ is used as the conditional vector c .

Before reading further, the reader is encouraged to have a look at Appendix B which includes a brief introduction to deep learning and explains the terms used to describe the architecture of the models.

2.2.1 Model Architecture

GAN-CLS uses a deep convolutional architecture for both the generator and the discriminator, similar to DC-GAN (Deep Convolutional-GAN) [27].

In the generator, a noise vector z of dimension 128, is sampled from $\mathcal{N}(\mathbf{0}, \mathbf{I})$. The text t is passed through the function ϕ and the output $\phi(t)$ is then compressed to dimension 128 using a fully connected layer with a leaky ReLU activation. The result is then concatenated with the noise vector z . The concatenated vector is transformed with a linear projection and then passed through a series of deconvolutions with leaky ReLU activations until a final tensor with dimension $64 \times 64 \times 3$ is obtained. The values of the tensor are passed through a tanh activation to bring the pixel values in the range $[-1, 1]$.

In the discriminator, the input image is passed through a series of convolutional layers. When the spatial resolution becomes 4×4 , the text embeddings are compressed to a vector with 128 dimensions using a fully connected layer with leaky ReLU activations as in the generator. These compressed embeddings are then spatially replicated and concatenated

in depth to the convolutional features of the network. The concatenated tensor is then passed through more convolutions until a scalar is obtained. To this scalar, a sigmoid activation function is applied to bring the value of the scalar in the range $[0, 1]$ which corresponds to a valid probability.

The focus of the GAN-CLS paper is not on the details of the architecture of the discriminator and the generator. Thus, to obtain better results, I deviated slightly from the DC-GAN architecture, and I added one residual layer [11] in the discriminator and two residual layers in the generator. These modifications increase the capacity of the networks and lead to more visually pleasant images. Figure 2.1 shows the architecture of the customised GAN-CLS.

2.2.2 Adapting GAN Loss to Text-Image Matching

The GAN-CLS critic has a slightly different loss function from the one presented in Equation 1.3. The goal of the modification is to better enforce the text-image matching by making the discriminator to be text-image matching aware. The critic cost function from Equation 2.1 is used.

$$L_D = -\mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mat}} [\log(D(\mathbf{x}, \mathbf{e}))] - \frac{1}{2} (\mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{ge}} [\log(1 - D(\mathbf{x}, \mathbf{e}))] + \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mis}} [\log(1 - D(\mathbf{x}, \mathbf{e}))]) \quad (2.1)$$

where $\mathbb{P}_{r-mat}$ is the joint distribution of image and text embeddings which match, $\mathbb{P}_{r-mis}$ is the joint distribution of image and text embeddings which mismatch, $\mathbb{P}_{ge}$ is the joint distribution of generated images and the text embeddings they were generated from and $\mathbf{e} = \phi(\mathbf{t})$ is a text embedding. In this way, the discriminator has a double functionality: distinguishing real and fake images but also distinguishing between the text-image pairs which match and those which mismatch.

This flower is pink and white in colour, with petals that are multicoloured.



The flower shown has bright purple petals with white pistil in the center.



Figure 2.2: Samples generated from text descriptions from the test dataset. For each text description, the model generates multiple samples, each using a different input noise vector. As it can be seen the model ignores the input noise and the resulting images are extremely similar. The disappearance of the stochastic behaviour is a current research problem in Conditional GANs.

2.2.3 Training

Adam optimiser [18] is used for training. Adam maintains a separate learning rate for each of the parameters of the model and uses a moving average of the first and second moment of the gradients in the computation of the parameter update. The usage of the gradient statistics makes the algorithm robust to gradient scaling and well suited for problems with noisy gradients such as this one. The parameters β_1 and β_2 control the decay rate of these moving averages. The learning rate is set to 0.0002 for both networks, $\beta_1 = 0.5$ and $\beta_2 = 0.9$. The model is trained for a total of 600 epochs with a batch size of 64.

2.2.4 Results

Figure 2.2 shows samples generated for the flowers dataset. All the shown samples are produced from descriptions from the test dataset.

This flower is white and pink in colour, with petals that are spotted.



The flower has four petals, one petal is purple and the others have yellow and red stripes.

The flower is pink in colour, with petals that are wavy.



The petals on this flower are yellow with no visible stamen.

Figure 2.3: Interpolations in the conditional embedding space while maintaining the noise vector constant. The top description for each image corresponds to the image on the left and the bottom description corresponds to the image on the right. The images in between are generated from interpolations between these two descriptions.

A common way to test that the models learns the visual semantics of the text descriptions and it does not merely memorise the description-image mappings is to generate images $G(z, (1 - t)e_1 + te_2)$ from interpolations between two text embeddings e_1 and e_2 where t is increased from 0 to 1. If the model works, these transitions should be smooth. Figure 2.3 shows images produced by GAN-CLS from such interpolations.

2.3 Stacked GANs

One would rarely see an artist producing a painting in full detail directly from the first attempt. By analogy, this is how GAN-CLS described in section 2.2 generates images. These architectures do not usually scale up well to higher resolutions. It would be desired

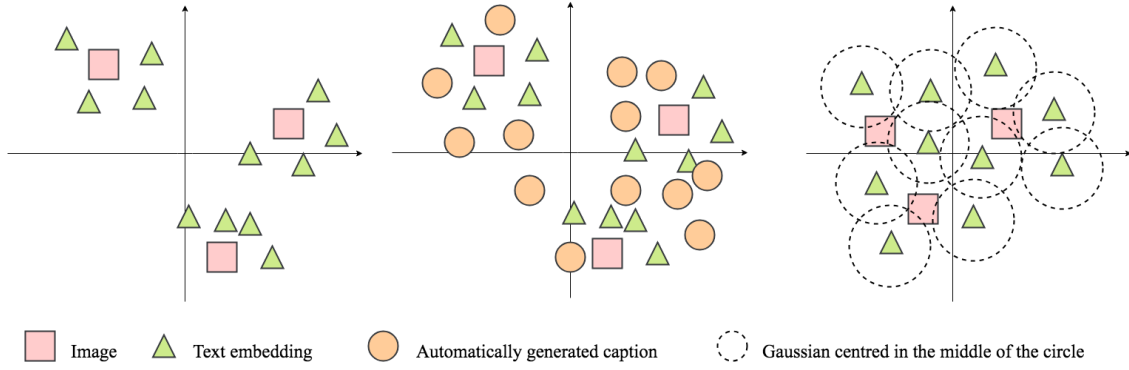


Figure 2.4: Illustration of a simplified 2D conditional space before augmentation (left) and after adding two different augmentation strategies (middle and right). The image captioning system from [6] fills the embedding space with synthetic captions (middle). The conditional augmentation from StackGAN [37] ensures a smooth conditional manifold by sampling from a Gaussian distribution for each text embedding (right)

to have a network architecture which is closer to the analogy of a painter who starts with the main shapes, colours, textures and then gradually adds details.

StackGAN [37] is such an architecture, and it uses two GANs. The first GAN, called Stage I, generates images from captions at a lower resolution of 64×64 in a similar manner to GAN-CLS. The second GAN, called Stage II, has a generator which takes as input the image generated by the Stage I generator and produces a higher resolution 256×256 image with more fine-grained details and better text-image matching.

2.3.1 Text Embedding Augmentation

Besides this generation of images at multiple scales, the StackGAN paper proposes the augmentation of the conditional space. Because the number of text embeddings is small, they cover tiny, sparse regions in the embedding space clustered around their corresponding images. The model hardly understands the visual semantics of the embeddings at test time because these embeddings are usually far in the embedding space from the ones seen in training. Moreover, the low number of embeddings is likely to cause the model to overfit.

Dong et al. [6] have also independently recognised this problem. They propose an image captioning system to fill the embedding space. Nevertheless, this is far from an ideal solution. The curse of dimensionality takes effect, and it is unfeasible to fill the space in such a manner. Moreover, the image captioning system adds significant computational costs.

StackGAN uses another approach inspired by another generative model, Variational Autoencoders (VAE) [19]. For a given text embedding $\phi(\mathbf{t})$, augmented embeddings can be sampled from a distribution $\mathcal{N}(\mu(\phi(\mathbf{t})), \text{Cov}(\phi(\mathbf{t})))$. As in VAEs, to ensure that the conditional space remains smooth and the model does not overfit, a regularisation term enforces a standard normal distribution over the normal distributions of the embeddings. The regularisation term with hyper-parameter ρ (Equality 2.2) consists of the KL divergence between the normal distribution of the embeddings and the standard normal

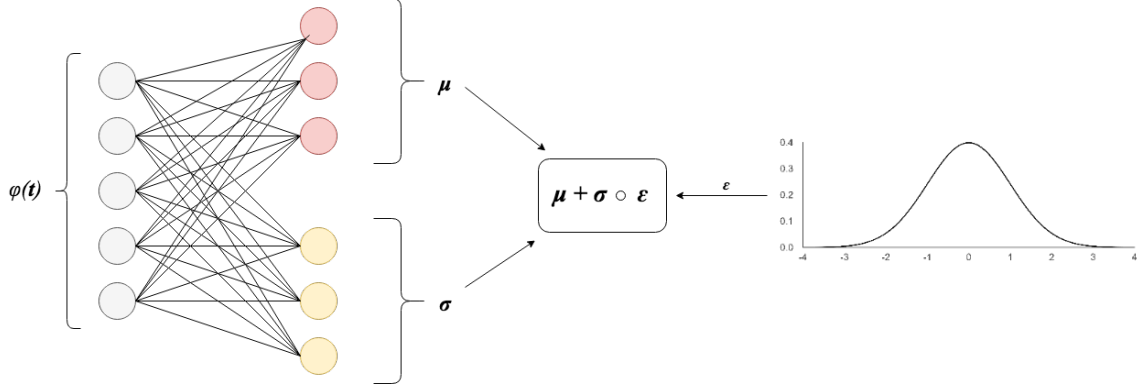


Figure 2.5: The VAE reparametrization trick. The network learns μ and σ and uses a sampled ϵ to compute an augmented embedding. The associated sampling noise causes improved image variation as the model generates different images for different samples of the same embedding.

distribution.

$$L_G = -\mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g, \mathbf{T} \sim \mathbb{P}_r} [\log(D(\mathbf{x})) + \rho KL(\mathcal{N}(\mathbf{0}, \mathbf{I}) \parallel \mathcal{N}(\mu(\phi(\mathbf{t})), \Sigma(\phi(\mathbf{t})))], \quad (2.2)$$

The reparametrization trick from VAEs is used to perform the sampling. With this trick, the network has the independence to learn the mean μ and the standard deviation σ of the embedding. For an embedding $\mathbf{e} = \phi(\mathbf{t})$, a fully connected layer with leaky ReLU activations computes μ and another computes σ and the sampled vector $\hat{\mathbf{e}}$ is obtained as shown in Equation 2.3 (Figure 2.5).

$$\hat{\mathbf{e}} = \mu + \sigma \circ \epsilon, \text{ where } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ and } \circ \text{ is element-wise multiplication} \quad (2.3)$$

2.3.2 Model Architecture

Figure 2.6 shows the full architecture of StackGAN. The architecture of the Stage I generator is identical to the one of the customised GAN-CLS (described in Section 2.2.1) with the addition of the conditioning augmentation (CA) module previously discussed.

The Stage II generator starts by down-sampling the input 64×64 image until it reaches a spatial resolution of 4×4 . To this 4×4 block, the corresponding augmented text embedding is concatenated in depth to improve the text-image matching of Stage I. The concatenated block is passed through three residual layers and then up-sampled until a final tensor of $256 \times 256 \times 3$ is obtained. In the end, tanh activation is applied to bring the output in $[-1, 1]$.

The Stage I discriminator is identical to the customised GAN-CLS discriminator previously discussed. The Stage II discriminator is also similar, with the exception that more down-sampling convolutional layers are used to accommodate for the higher resolution of the input.

As in the paper, ReLU activations are used for the generator and leaky ReLU activations for the discriminator. Batch normalisation is applied both in the generator and the discriminator.

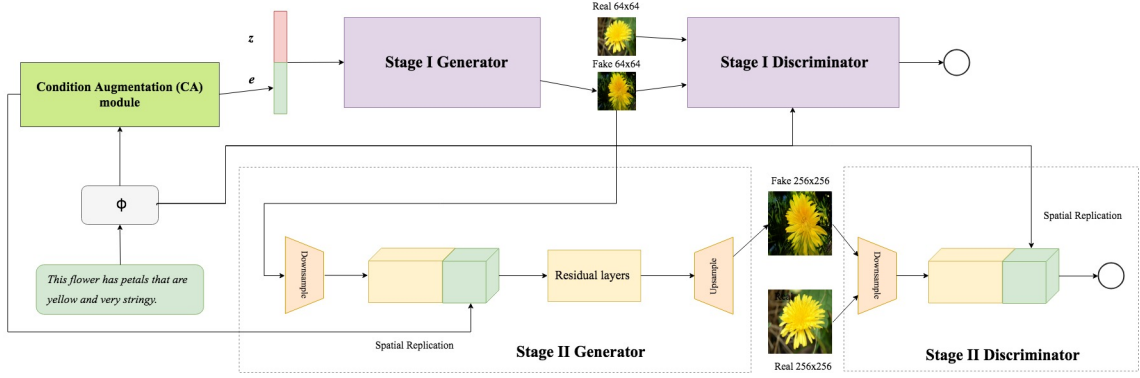


Figure 2.6: The architecture of StackGAN. The architecture of Stage I is identical to the customised GAN-CLS presented in the previous section. The Stage II generator takes as input and fine-tunes the image generated by Stage I. The generators of both stages use the augmented embeddings.

This flower has petals that are overlapping and dark orange with yellow center and stamen.



A flower with long pink petals and raised orange stamen.



Figure 2.7: Samples generated by Stage I of StackGAN.

2.3.3 Training

StackGAN uses the same discriminator loss function as the one in 1.3 and the generator loss from 2.2.

For training, I used the Adam optimiser with a learning rate of 0.0002 for both networks, $\beta_1 = 0.5$ and $\beta_2 = 0.9$. I trained each of the stages for 600 epochs using a batch size of 64 for Stage I and a batch size of 32 for Stage II. When training Stage II, the parameters of Stage I are no longer trained. The learning rate is halved every 100 epochs as recommended in the paper.

2.3.4 Results

Figure 2.7 shows samples generated by Stage I and Figure 2.9 includes samples created by Stage II. The stochastic behaviour introduced by the augmentation of the text embeddings reflects in the higher image diversity of the generated images. Conditional interpolations for Stage I and Stage II are shown in Figures 2.8 and 2.10. Figure 2.11 shows the images produced by the two stages for the same descriptions. Images generated by StackGAN on the birds dataset are included in Appendix D.

A flower with long pink petals and raised orange stamen.



A vivid yellow flower with spiked looking petals and a darker yellow center.

The flower shown has blue petals and blue pistil as well.



The flower has large white and yellow petals.

Figure 2.8: Samples generated by Stage I of StackGAN from text embedding interpolations.

The flower shown has smooth white petals with yellow patches as well.



This small flower has an out row of tiny long petals and yellow middle of stamen.

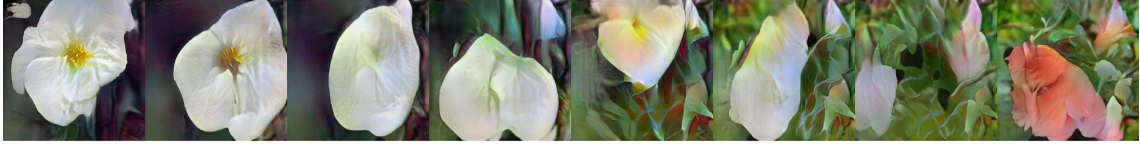


Figure 2.9: Samples generated by Stage II of StackGAN.

2.4 Other Models

Other two state of the art models have been proposed since the start of this project: StackGAN-v2 [38], and more recently, AttnGAN [36] developed by Microsoft Research in collaboration with other universities. StackGAN-v2, as the name suggests, is an improved version of StackGAN which uses multiple generators and discriminators in a tree-like structure. AttnGAN consists of an architecture similar to StackGAN-v2, but with an attention model [4, 23] on top of it. The attention model replicates the human attention mechanism and allows the network to focus on a single word from a sentence or a specific region of the image at a time. This ensures a granular image-word matching and not just a sentence level matching as it is the case with the other models discussed in this work.

This white flower has a single white petal which is long and pointer and wraps around a yellow pistil.



This flower is pink in colour, with petals that are bell shaped.

This fierce flower has long pointy petals that are orange in colour and sharp, black stigma.



These flowers have thin and orange petals that grow vertically from each other.

Figure 2.10: Samples generated by Stage II of StackGAN from text embedding interpolations.



Figure 2.11: Images generated by Stage I (first row) and Stage II (second row) for the same text descriptions (one for each column). Stage II fine-tunes the images generated by Stage I.

Chapter 3

Research

In this chapter, I propose new models which try to address some of the current research problems. In Section 3.1 I propose Wasserstein GAN-CLS, a conditional Wasserstein GAN based on the recently introduced Wasserstein distance which offers convergence and stability guarantees. This model uses a novel loss function which achieves the text-image conditioning using a Wasserstein distance. In Section 3.2 I propose a conditional Progressive GAN inspired from [17] which learns to generate images at iteratively increasing scale, and I show how the conditional Wasserstein loss improves this model.

3.1 Wasserstein GAN-CLS

This section is more mathematical, in line with the firm theoretical arguments behind Wasserstein GANs. Additional explanations are included in Appendix C.

The main problem of GANs is their instability during training. Perhaps counter-intuitively, as the discriminator becomes better, the generator's updates get worse. Arjovsky et al.[2] show that this problem is related to how the distances $d(\mathbb{P}_r, \mathbb{P}_g)$, which GANs commonly optimise, behave when the support of $\mathbb{P}_r$ and $\mathbb{P}_g$ are disjoint or lie on low dimensional manifolds. When that is the case, a perfect discriminator which separates them always exists. As the discriminator approaches optimality, the gradient of the generator becomes unstable if the generator uses the loss function L_G from 1.3.

In many situations, it is likely that the two distributions lie on low dimensional manifolds. In the case of natural images, there is substantial evidence that the support of $\mathbb{P}_r$ lies on a low dimensional manifold [24]. Moreover, Arjovsky et al. [2] prove that this is the case with $\mathbb{P}_g$ in the case of GANs. Thus, the choice of the distance $d(\mathbb{P}_r, \mathbb{P}_g)$ is crucial. One would like this function to be continuous and provide non-vanishing gradients that can be used for backpropagation even when this situation occurs.

Maximum likelihood models implicitly optimise $KL(\mathbb{P}_r \parallel \mathbb{P}_g)$ (which is not a distance in the formal sense). GANs implicitly optimise the Jensen-Shannon divergence $JS(\mathbb{P}_r \parallel \mathbb{P}_g)$, as shown in the proof of Theorem 1.3.1 from Appendix A. Both of them are problematic. A simple example of two distributions whose supports are parallel lines [2] shows not only that these divergences (and others) are not differentiable, but they are not even

continuous. In the next section, I discuss the Wasserstein distance which was proposed as a better choice for $d(\mathbb{P}_r, \mathbb{P}_g)$.

3.1.1 Wasserstein GAN

The Wasserstein distance, also known as the Earth Mover's (EM) distance, is theoretically proposed and analysed in GANs for the first time in [2]. In [3] it is shown in practice that a GAN which optimises the Wasserstein distance offers convergence and stability guarantees while producing good looking and more diverse images. This distance is given in Equation 3.1.

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(\mathbf{X}, \mathbf{Y}) \sim \gamma} [\|\mathbf{x} - \mathbf{y}\|], \quad (3.1)$$

where $\Pi(\mathbb{P}_r, \mathbb{P}_g)$ is the set of joint distributions which have $\mathbb{P}_r$ and $\mathbb{P}_g$ as marginals and $\gamma_{\mathbf{X}, \mathbf{Y}}(\mathbf{x}, \mathbf{y}) \in \Pi(\mathbb{P}_r, \mathbb{P}_g)$ is one such distribution (see Figure 3.1 for an intuitive explanation).

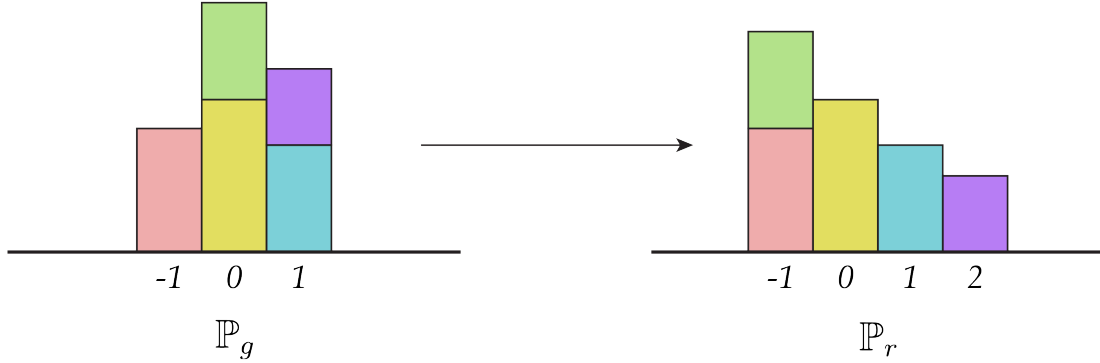


Figure 3.1: A better intuition for the Wasserstein distance can be developed by analysing a small discrete case. Given two discrete distributions $\mathbb{P}_r$ and $\mathbb{P}_g$, the Earth Mover's distance is the cost of the optimal plan to transport blocks of $\mathbb{P}_g$ to obtain $\mathbb{P}_r$ (or the other way around). A transport plan is optimal if it has minimum effort. The effort is proportional to the size of the blocks which are moved and the distance on which they have to be moved.

Of course, computing the Wasserstein distance in the form 3.1 is intractable. Nevertheless, its dual form is tractable. This form is given by the Kantorovich-Rubinstein duality [34] presented in Equation 3.2.

$$W(\mathbb{P}_r, \mathbb{P}_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} [f(\mathbf{x})] - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} [f(\mathbf{x})], \quad (3.2)$$

where the supremum is taken over the set of functions $f : \mathcal{X} \rightarrow \mathbb{R}$ which are 1-Lipschitz continuous (explained in Appendix C). Instead of optimising relation 3.2 in function space, it can be optimised in the space of parametric functions using a neural network $D_\omega : \mathcal{X} \rightarrow \mathbb{R}$. Equality 3.2 can be rewritten as:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \max_{\omega, \|D_\omega\|_L \leq 1} \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} [D_\omega(\mathbf{x})] - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} [D_\omega(\mathbf{x})] \quad (3.3)$$

The only question which remains is how to enforce the Lipschitz constraint. It was originally suggested [3] to keep the weights $\omega \in \mathcal{W}$, where $\mathcal{W}$ is a compact space such as $\mathcal{W} = [0.01, 0.01]^{N_w}$ and N_w is the number of weights. Keeping the weights in such a small range indirectly constraints the rate of growth of the function which remains K-Lipschitz continuous over the course of training. The exact value of K depends only on the choice of $\mathcal{W}$ and is independent of the values of ω . Nevertheless, this does not fully solve the problem because the small weights diminish the capacity of the neural network and also cause the training time to increase. A better solution was proposed [10] which softens the constraint by appending it to the loss function as a regularisation term.

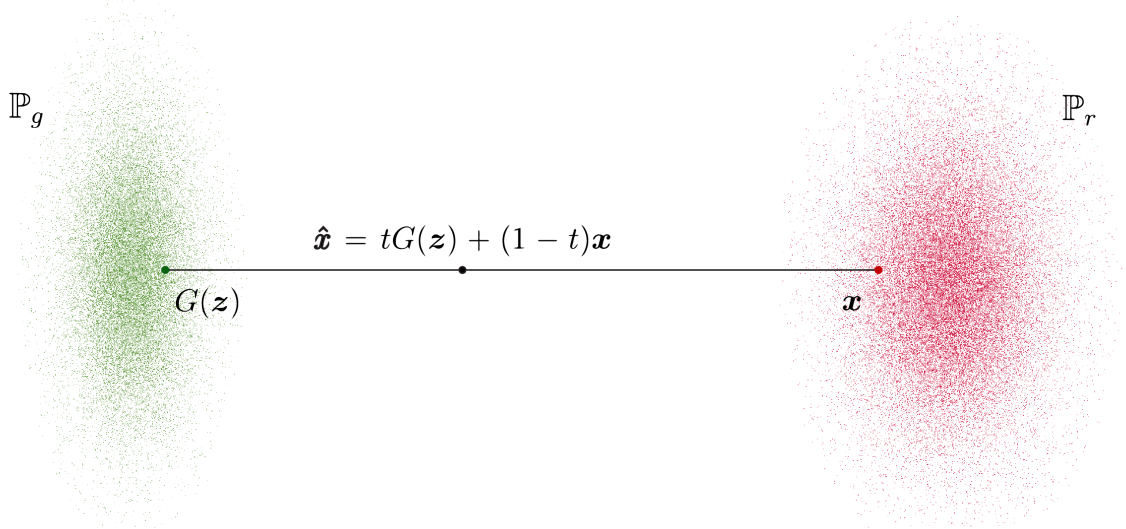


Figure 3.2: Linear interpolation of a point from the dataset and a generated image. The gradient penalty ensures that the gradient norm remains close to one for such points between the two distributions.

It can be shown that a differentiable function is 1-Lipschitz if and only if its gradient norm is at most one almost everywhere. This motivates the loss function from Equation 3.4 which adds a gradient penalty (L_{GP}) to penalise the network when the gradient norm goes far from one.

$$\begin{aligned} L_D &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g}[D(\mathbf{x})] - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r}[D(\mathbf{x})] + \lambda L_{GP} \\ &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g}[D(\mathbf{x})] - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r}[D(\mathbf{x})] + \lambda \mathbb{E}_{\hat{\mathbf{X}} \sim \mathbb{P}_{\hat{\mathbf{X}}}}[(\|\nabla D(\hat{\mathbf{x}})\| - 1)^2], \end{aligned} \quad (3.4)$$

where $\hat{\mathbf{x}}$ is a linear interpolation between a real and generated image: $\hat{\mathbf{x}} = tG(\mathbf{z}) + (1-t)\mathbf{x}$ and t is sampled from $U[0, 1]$. The model uses these interpolations because it is intractable to enforce the gradient constraint over the whole space $\mathcal{X}$. Instead, it is enforced only over the region between the two manifolds of the two distributions (Figure 3.2).

On the one hand, the discriminator is trained to better approximate the Wasserstein distance. The generator, on the other hand, tries to minimise $W(\mathbb{P}_r, \mathbb{P}_g)$, so the loss function from 3.5 is employed.

$$L_G = -\mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g}[D(\mathbf{x})], \quad (3.5)$$

3.1.2 Conditioning Wasserstein GAN

The loss function 3.4 makes the discriminator distinguish between real and fake samples, but for text to image synthesis, it must also be text-image matching aware. By making the discriminator to also approximate $W(\mathbb{P}(\mathbf{x}, \mathbf{e})_{r-mat}, \mathbb{P}(\mathbf{x}, \mathbf{e})_{r-mis})$ between the joint distributions of matching and mismatching text-image pairs, the discriminator becomes matching aware. Based on this insight, I propose the loss function from 3.6 for the discriminator.

$$\begin{aligned} L_D &= \{\mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{ge}}[D(\mathbf{x}, \mathbf{e})] - \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mat}}[D(\mathbf{x}, \mathbf{e})]\} \\ &\quad + \alpha \{\mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mis}}[D(\mathbf{x}, \mathbf{e})] - \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mat}}[D(\mathbf{x}, \mathbf{e})]\} \\ &= \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{ge}}[D(\mathbf{x}, \mathbf{e})] + \alpha \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mis}}[D(\mathbf{x}, \mathbf{e})] \\ &\quad - (1 + \alpha) \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mat}}[D(\mathbf{x}, \mathbf{e})] + \lambda L_{LP} \end{aligned} \quad (3.6)$$

where the parameter α controls the level of text-image matching.

Note that another regularisation term (L_{LP}), different from L_{GP} , is used to enforce the Lipschitz constraint. A potential problem of this loss function is that it can take values with a high magnitude on some datasets or architectures. Because nothing minimises $W(\mathbb{P}_{r-mat}, \mathbb{P}_{r-mis})$ as it is the case with $W(\mathbb{P}_{r-mat}, \mathbb{P}_{ge})$ which is being minimised by the generator, $W(\mathbb{P}_{r-mat}, \mathbb{P}_{r-mis})$ can theoretically take very high values. High values of this distance can damage the gradient penalty term whose proportion in the loss function will become so small that the gradient norm will get out of control. Theoretically, this can be fixed by simply increasing λ , but the regularisation of WGAN-GP from 3.4 (L_{GP}) is not so robust [26] to changes in the values of the parameter λ . To address this, I use instead the regularisation term recently proposed in [26] which is called L_{LP} (LP - Lipschitz Penalty). This term which does not penalise gradient norms less than one allows for larger values of λ without harming the model. Moreover, empirical and theoretical evidence [26] shows that, under this softer regularisation term, convergence is faster and more stable. L_{LP} is given by:

$$L_{LP} = \mathbb{E}_{(\hat{\mathbf{x}}, \mathbf{e}) \sim \mathbb{P}_\eta} [\max(0, \|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}}, \mathbf{e})\| - 1)^2 + \max(0, \|\nabla_{\mathbf{e}} D(\hat{\mathbf{x}}, \mathbf{e})\| - 1)^2] \quad (3.7)$$

where I use $\mathbb{P}_\eta$ to denote the joint distribution of image text pairs $(\hat{\mathbf{x}}, \mathbf{e})$. $\hat{\mathbf{x}} = tG(\mathbf{z}, \mathbf{e}) + (1 - t)\mathbf{x}$ is a linear interpolation with t sampled from $U(0, 1)$ and $\mathbf{e}$ is a matching text embedding of the image $\mathbf{x}$. Note that because $D(\hat{\mathbf{x}}, \mathbf{e})$ is also a function of the text embeddings $\mathbf{e}$ in this case, the Lipschitz constraint needs to be enforced with respect to the input $\mathbf{e}$ as well, not only $\hat{\mathbf{x}}$, hence the second term of the summation.

Regarding the generator, I use the same cost function as the one in 3.5 with the addition of the text augmentation loss which softly maintains the standard normal distribution over the conditional latent space as described in Section 1.3.2. Thus, the loss of the generator is:

$$L_G = -\mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_g} [D(\mathbf{x}, \mathbf{e})] + \mathbb{E}_{\mathbf{T} \sim \mathbb{P}_r} [\rho KL(\mathcal{N}(\mathbf{0}, \mathbf{I}) \parallel \mathcal{N}(\mu(\phi(\mathbf{t})), \Sigma(\phi(\mathbf{t}))))] \quad (3.8)$$

3.1.3 Architecture

To make comparisons simpler, I keep the architecture of the generator identical to that of Stage I of StackGAN. In the case of the discriminator, I remove the batch normalisation for the gradient penalty to work. The gradient penalty assumes a unique gradient for each sample, and this assumption no longer holds in the presence of batch normalisation [10].

Because, when the Wasserstein distance is used, the discriminator no longer needs to be crippled to keep the training balanced, I add one more convolutional layer in the discriminator after the text embedding concatenation, and I use the same number of convolutional filters as in the generator.

3.1.4 Training

In the case of Wasserstein GANs, the closer the discriminator gets to optimality for a fixed G , the better the approximation of $W(\mathbb{P}_{r-mat}, \mathbb{P}_{ge})$ and $W(\mathbb{P}_{r-mat}, \mathbb{P}_{r-mis})$ is. The generator’s updates will also be better. That is why the discriminator is trained for n_{critic} times for every generator update, where n_{critic} is a hyper-parameter to be set at training time. A common value is $n_{critic} = 5$.

This flower has white petals and yellow pistil as its main features.



The flower has four petals, one petal is purple and the others have yellow and red stripes.



Figure 3.3: Samples generated by WGAN-CLS from the same text descriptions but different noise vectors

For faster training, I take a slightly different approach by setting $n_{critic} = 1$ in conjunction with the usage of the Two-Timescale Update Rule (TTUR) [15]. TTUR refers to the usage of two different learning rates for the generator and the discriminator, which guarantees that the networks will reach a local Nash equilibrium. The convergence under TTUR is shown to be faster and the quality of the images higher than in the case of the classic method of training. Thus, I use the Adam optimiser again with a learning rate of 0.0003 for the critic and 0.0001 for the generator. In the case of the other parameters of Adam, I use $\beta_1 = 0$ and $\beta_2 = 0.99$ for both the generator and the discriminator. The generator regularisation parameter ρ is set to 10, $\lambda_{LP} = 150$ and $\alpha = 1$. The training is performed for 120,000 steps with a batch size of 64.

3.1.5 Results

Figure 3.3 shows samples generated by the Conditional Wasserstein GAN. Figure 3.4 shows images generated from interpolations in the conditional space.

A flower with mid-sized orange petals and central cluster of yellow stamen.



This flower has a single white petal and white stamen.

A light pink flower with spaced petals on the outer edge and a large stigma.



This flower has a white petal and one long yellow stigma.

Figure 3.4: Samples generated by the Conditional Wasserstein GAN from interpolations between two text embeddings.

I subjectively assess that the quality of the generated images is comparable to the one of GAN-CLS and Stage I of StackGAN. My subjective evaluation is also confirmed by the Inception Scores of the models which are given in Chapter 4.

3.2 Conditional Progressively Growing GANs

In this section, I show how the recently introduced Progressive Growing GAN (PGGAN) [17] can be turned into a conditional model for text to image synthesis. Moreover, I show how the Wasserstein critic loss I proposed in the previous section can improve this conditional model.

3.2.1 Architecture and Training

Because the training of this model is tightly integrated with its architecture, I treat them together in this section rather than separately.

The generator starts by concatenating a noise vector $\mathbf{z}$ with an augmented embedding $\mathbf{e}$. This concatenated vector is then projected into a tensor of dimension $4 \times 4 \times 512$ which is then followed by two more convolutional layers with a filter size of 3×3 . Together, they constitute the first stage of the generator. The output of this stage is supplied as input to a stack of other stages, all separated by a nearest neighbour upsampling layer which upscales by a factor of two. All the generator stages excepting the first are composed of two convolutional layers with a kernel size of 3×3 . In the end, the output tensor is

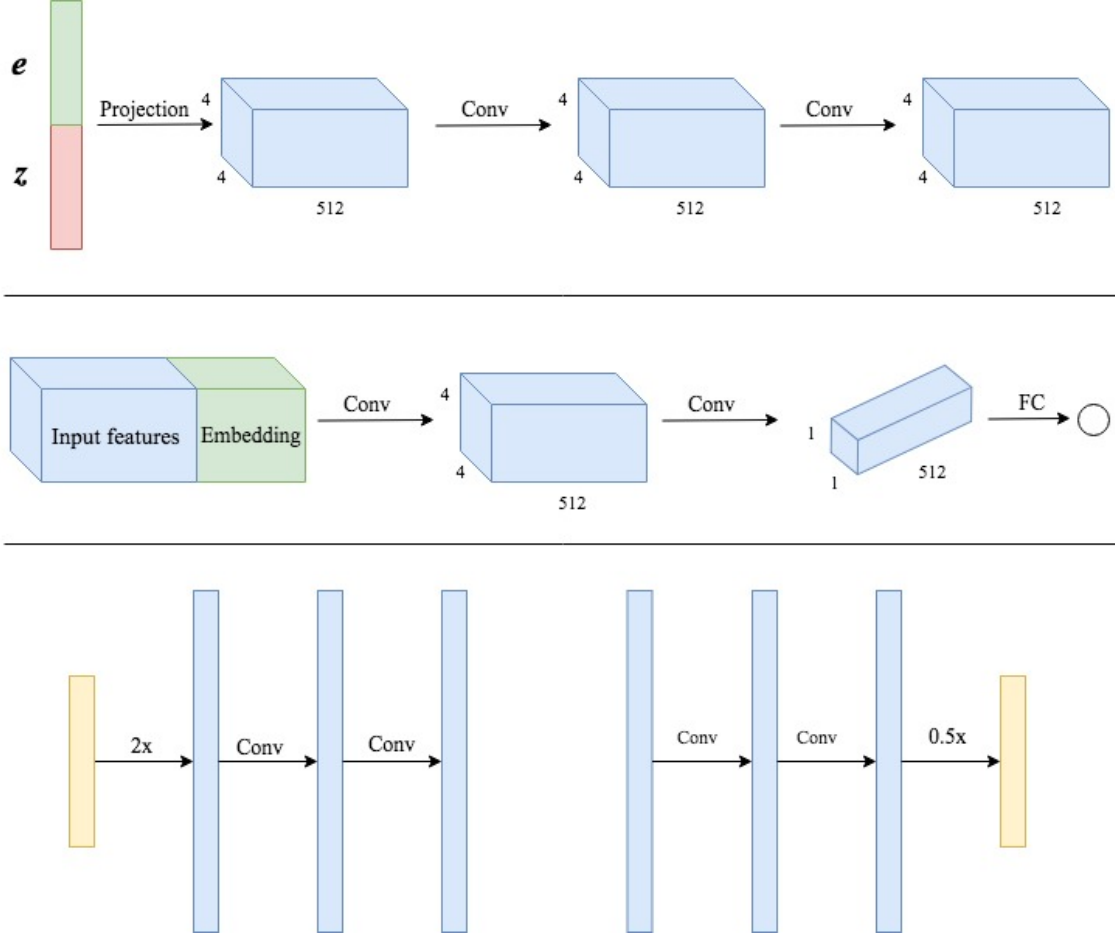


Figure 3.5: Stages of the conditional PGGAN: (Top) First stage of the generator, (middle) First stage of the discriminator, (bottom left) the architecture of the other stages of the generator, (bottom right) the architecture of the other stages of the discriminator.

passed through two more convolutional layers called the “toRGB” module which outputs the actual RGB image.

The discriminator starts with a convolutional layer which produces a first set of convolutional features without affecting the spatial resolution. This layer is denoted as the “fromRGB” module. These features are given as input to a stack of stages which again are added step by step as training progresses concurrently with the generator stages. All stages have two convolutional layers, symmetric to the ones of the generator. The only exception is the first stage where the compressed embeddings are concatenated in depth to the input features of that stage similarly to the previous GANs. The concatenated block is processed by two more convolutional layers followed by a fully connected layer which produces the scalar discriminator output. The discriminator stages are separated by an average pooling layer which reduces the resolution by a factor of two. Figure 3.5 shows the structure of all the stages.

The novelty of the model consists in the way the networks transform during training. The first stages of both networks are trained first using images of 4×4 in resolution. Then the second stage is introduced concurrently for the discriminator and the generator as

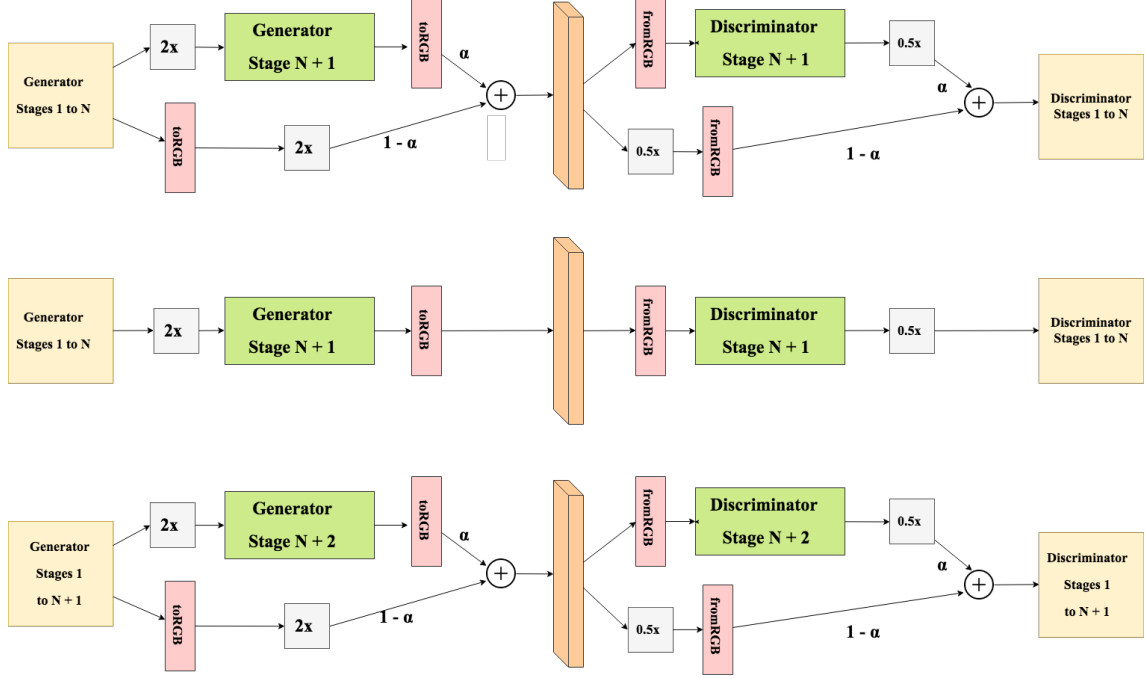


Figure 3.6: Three consecutive training phases for the conditional PGGAN: the transition phase of an arbitrary $N + 1$ stage (top), the stabilisation phase of the same stage (middle), the transition phase of stage $N + 2$. The parameter α is linearly increased from zero to one during the transition phase. At the end of the transition phase when $\alpha = 1$, the new stages are fully attached to the previous stages. Next, the networks stabilise in their new configuration during the stabilisation phase. After the new stages are stabilised, the transition phase of the next stage begins.

a residual layer with an associated weight $\alpha = 0$ to avoid perturbing the network. This stage doubles the resolution to 8×8 . The addition of any new stage starts with a transition phase. During a transition phase, α linearly increases to one, and the model smoothly learns to adapt to the new stages and the enlarged image size. This weight has the effect of interpolating between the scaled output of the previous stage and the output of the new stage in the case of the generator. For the critic, it is an interpolation of the inputs. When the transition phase is over, a stabilisation phase follows to stabilise the network in its new configuration with $\alpha = 1$. Each transition and stabilisation phase lasts until the discriminator sees 600,000 real images and 600,000 generated images. This process of iteratively adding stages repeats until the desired resolution is reached or as long as the GPU memory and the resolution of the images in the dataset allow it. When scaling up to a new stage, all the stages are trained, including the previous ones. The training process is also shown and further explained in Figure 3.6. More details are given in Appendix E.

As shown in [17], this method of training is significantly faster than training the complete network from scratch because the majority of the training time is spent at the lower stages.

3.2.2 The Need for a Stable Loss Function

This architecture and unusual method of training do not work with any loss function given the instability of GANs. The PGGAN paper [17] empirically shows PGGAN working with a least squares loss and a Wasserstein distance loss. While the least squares loss [22] is empirically known to be more stable than the classic GAN loss, the Wasserstein loss has technical reasons behind which guarantee its stability.

For the least squares loss, as with the Wasserstein loss, the discriminator no longer outputs a probability, but an arbitrary real number. The generator and the discriminator optimise for making this real number close to some predefined labels a, b, c . The general form of the least squares loss is as follows:

$$\begin{aligned} L_D &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} [(D(\mathbf{x}) - b)^2] + \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} [(D(\mathbf{x}) - a)^2] \\ L_G &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} [(D(\mathbf{x}) - c)^2] \end{aligned} \quad (3.9)$$

The flower has large white and yellow petals.



A flower with large, multiple layered slender petals of bright yellow with a short stamen.



Figure 3.7: Samples generated by the Conditional Least Squares PGGAN.

As shown in [22], when $b - c = 1$ and $b - a = 2$, minimising these cost functions is equivalent to minimising the Pearson χ^2 divergence between $\mathbb{P}_r + \mathbb{P}_g$ and $2\mathbb{P}_g$. This justifies the choice of labels $a = -1, b = 1, c = 0$ which I use for my experiments. It is trivial to adapt this loss function to make the discriminator matching aware as follows:

$$\begin{aligned} L_D &= \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mat}} [(D(\mathbf{x}, \mathbf{e}) - b)^2] + \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{ge}} [(D(\mathbf{x}, \mathbf{e}) - a)^2] \\ &\quad + \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{r-mis}} [(D(\mathbf{x}, \mathbf{e}) - a)^2] \\ L_G &= \mathbb{E}_{(\mathbf{X}, \mathbf{E}) \sim \mathbb{P}_{ge}} [(D(\mathbf{x}, \mathbf{e}) - c)^2] \end{aligned} \quad (3.10)$$

Now, the discriminator will push towards a not only the synthetic images but also images which do not match their description.

Nevertheless, I find the least squares loss to be unstable when the network reached the high-resolution stages, which is consistent with the findings from [17]. As in the paper, I introduce multiplicative Gaussian noise between the layers of the discriminator to eliminate the instability. This hack does not address the cause of the problem, which is the loss function. The Conditional Progressive Growing GAN is a perfect use case for

This round flower has a yellow center surrounded by hundreds of very thin yellow petals.



This flower is circular in structure and has very dense, small purple petals.

The petals of this flower are purple with a long stigma.



This is a flower with pink, white and yellow petals on it.

Figure 3.8: Samples generated by the Conditional Least Squares PGGAN from text embedding interpolations.

the Wasserstein based loss I proposed in Section 3.1 because it is guaranteed to be stable. Results for both of these losses are discussed in the next section.

3.2.3 Results

Figure 3.7 includes 256×256 samples generated by the Conditional Least Squares PGGAN (CLSPGGAN). Figure 3.8 includes images generated by the same model from text embedding interpolations. Figures 3.9 and 3.10 include the equivalent images generated by the Conditional Wasserstein PGGAN (CWPGGAN) which use the Wasserstein loss I proposed in section 3.1.

Images generated by CWPGGAN on the birds dataset are included in Appendix D.

The petals of the flower are pink in colour and have a yellow center.



This flower has long semi pointed bright orange petals with a brown seedy like stamen.



Figure 3.9: Samples generated by the Conditional Wasserstein PGGAN

Figures 3.11 and 3.12 show the images generated by each stage of Conditional PGGAN for the Least Squares loss and the Wasserstein loss respectively.

This flower has long white petals and a large yellow stigma in the middle.



The flower shown has three rows of orange petals and a yellow center.

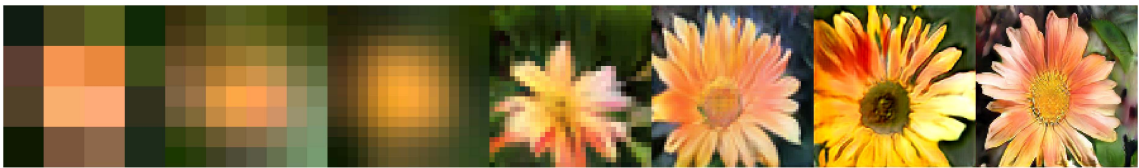
A vivid yellow flower with spiked looking petals and a darker yellow center



This flower has many purple stamen surrounded by long white petals with purple centers.

Figure 3.10: Samples generated by the Conditional Wasserstein PGGAN from interpolations between two text embeddings.

An orange flower with long, skinny petals and orange yellow center.



A flower with purple petals with no visible pistils or anther filaments.

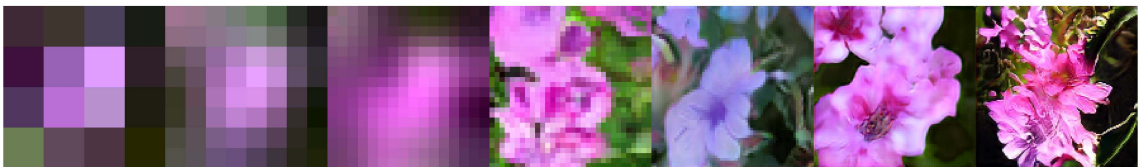


Figure 3.11: The image generated by each stage of the Least Squares Conditional PGGAN for the same text description. The images range from resolutions 4×4 to 256×256 . Each stage doubles the resolution.

This flower has a yellow center surrounded by long orange and yellow petals.



This flower is blue and white in colour, with petals that are wavy and ruffled.



Figure 3.12: The image generated by each stage of the Wasserstein Conditional PGGAN for the same text description. The images range from resolutions 4×4 to 256×256 . Each stage doubles the resolution.

Chapter 4

Evaluation and Comparisons

In this chapter, I present the Inception Score [30], the current standard technique for evaluating GANs. Then, I show the score for all of the models introduced in this work as well as a side by side comparison of the images they produce.

4.1 The Inception Score

The evaluation of generative models is a current area of research. Because most generative models maximise the likelihood of the data, they are evaluated using the average log-likelihood as a metric. As previously discussed, GANs depart from this approach and thus perform better, but at the same time, this also makes their evaluation harder. A recently proposed way of evaluating GANs which generate images is the Inception Score [30].

The name of the score comes from Google’s Inception classifier [31] (Figure 4.1). Treating images as a random vector $\mathbf{X}$ and the image labels as a random variable Y , the Inception network produces a distribution $P_{Y|\mathbf{X}}$ where $P_{Y|\mathbf{X}}(y|\mathbf{x})$ is the probability assigned to image $\mathbf{x}$ to belong to class y . An Inception network is trained to produce such probabilities for the classes from the test dataset the GAN will be evaluated on. This assumes a dataset divided into classes. Then, the trained network classifies the images generated by the model being evaluated. The score is a function of the distribution of the predicted classes. There are two desired outcomes:

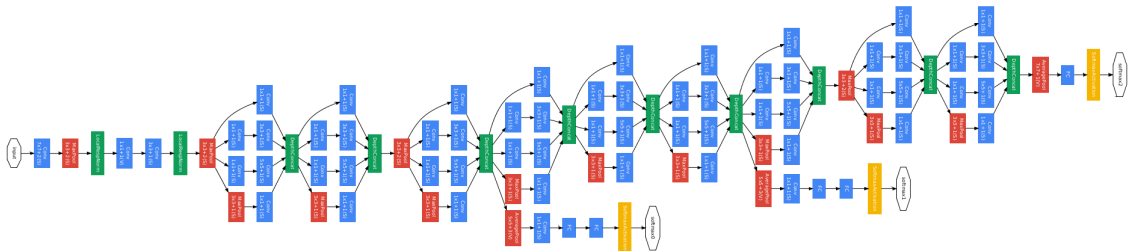


Figure 4.1: The architecture of the inception network (image taken from [31]). The bigger blue blocks are convolutions, the smaller blue blocks are fully connected layers, the red blocks are pooling layers, and the yellow blocks are softmax layers which convert the layer input values in a valid probability distribution. The two bottom branches which separate from the main part of the network are auxiliary classifiers, which are used for better gradient propagation.

1. The object in any image $\mathbf{x}$ should be undoubtedly identifiable. In other words, the conditional distribution $P_{Y|\mathbf{X}}$ should have low entropy.
2. All the generated images should be as diverse as possible. That is, the images should not belong to just a small subset of classes but all the classes in the dataset. Equivalently, the distribution P_Y should have high entropy.

These two aspects motivate the form of the Inception Score from 4.1 because if they hold, then the KL divergence between the two mentioned distributions is high. The exponential function is used only for aesthetic reasons to bring the values of the score in a higher range of values.

$$IS(G) = \exp(\mathbb{E}_{\mathbf{X} \sim P_g}[KL(P_{Y|\mathbf{X}}(y|\mathbf{x}) \parallel P_Y(y))]) \quad (4.1)$$

The Inception score was shown to correlate well with human evaluation of image quality [30]. This is the reason I chose not to conduct a human assessment for evaluating the models presented in this work.

For training I use the Inception-V3 architecture [32], a variant of the architecture shown in Figure 4.1. Instead of fully training the network, I only fine tune it. I train only the “Logits” and “Mixed_7c” variable scopes and for the other layers, I use the publicly available weights trained on ImageNet [5]. This follows the approach from StackGAN [37]. For computing the Inception Score, I use a group of 50,000 generated images which are split randomly into ten equal sets as recommended in [30]. The inception score is computed for each of these sets and the mean value together with the standard deviation are reported.

4.1.1 Evaluation of Text-Image Matching

The Inception score in its default form measures only the image quality, but it can also be used as an implicit measure of text-image matching. The Inception network is trained on the test dataset which (very importantly) contains classes disjoint from those in the training dataset. The generated images which are evaluated are produced exclusively from text descriptions from the test dataset. Because the training and test datasets contain disjoint classes, neither the text descriptions nor the images from the test dataset (or similar ones) are seen by the model in training. To generate high Inception Scores, the model must create images similar to the ones from the test dataset. The only possibility for the model to do this is to learn the visual semantics of the text descriptions correctly and to generate high-quality images which respect those descriptions. Thus, the reported Inception-Score is a measure of both image quality and text-image matching. The StackGAN paper [37] uses the same approach in its evaluation.

4.2 Inception Score Results

I include the Inception Score means and standard deviations for all models on the Oxford-102 flowers dataset in Table 4.1. The results show that the proposed Conditional Wasserstein GAN obtains comparable results to other state of the art models which produce 64×64 images while maintaining the same generator as Stage I of StackGAN. Moreover, the Conditional Wasserstein GAN achieves this score in conditions of guaranteed training stability which is very important. The proposed Conditional Progressive Growing GAN achieves a better score than the other models for both resolutions on the flowers dataset. Moreover, the model obtains the best score in combination with the Wasserstein loss I proposed in Section 3.1.

Model	Resolution	Score
Customised GAN-CLS	64×64	3.11 ± 0.03
StackGAN Stage I	64×64	3.42 ± 0.02
WGAN-CLS*	64×64	3.11 ± 0.02
WGAN-CLS with TTUR*	64×64	3.20 ± 0.01
CLSPGGAN*	64×64	3.44 ± 0.04
CWPGGAN*	64×64	3.70 ± 0.03
StackGAN Stage II	256×256	3.71 ± 0.04
CLSPGGAN*	256×256	3.76 ± 0.03
CWPGGAN*	256×256	3.86 ± 0.02

Table 4.1: Inception Scores for the Oxford-102 flowers dataset. Models marked with * are the models proposed in this report.

On the birds dataset, I run limited experiments for CWPGGAN and StackGAN (Appendix D). To quickly evaluate CWPGGAN against the other models, including the recently introduced StackGAN-v2 and AttnGAN, I used directly the scores given in the AttnGAN paper [36] for the birds dataset. The flowers dataset is not used in the paper.

Thus, I computed the Inception score of CWPGGAN using the same (publicly available) Inception network used in the evaluation part of the AttnGAN paper. The score obtained by CWPGGAN, as well as the score of the other models, are given in Table 4.2.

Model	Resolution	Score
GAN-INT-CLS	64×64	2.88 ± 0.04
CWPGGAN*	64×64	3.18 ± 0.03
StackGAN	256×256	3.70 ± 0.04
StackGAN-v2	256×256	3.82 ± 0.06
CWPGGAN*	256×356	4.09 ± 0.03
AttnGAN	256×256	4.36 ± 0.04

Table 4.2: Inception scores for the CUB-200-2011 birds dataset using the Inception network used for evaluation in [36]. The Inception scores of all models, excepting CWPGGAN, are taken directly from [36]. Models marked with * are the models proposed in this report.

CWPGGAN boosts by 7.07% the best Inception Score on the birds dataset of the models which use only the sentence-level visual semantics. Moreover, CWPGGAN has

the second best Inception Score for 256×256 images out of all the existent state of the art models. The score of CWPGGAN and the quality of the images it produces is particularly impressive given the that it does not use any word-level visual semantics such as AttnGAN. This score is also achieved in conditions of guaranteed stability given by the proposed loss function in WGAN-CLS. Because AttnGAN is composed of a StackGAN-v2 with an attention model on top, these results are an indication for future research that CWPGGAN equipped with a similar attention model could produce even higher scores.

The results also prove that the proposed Wasserstein loss makes possible the usage of innovative architectures and training techniques which would not work with the standard loss function used by the existent text to image models.

4.3 Side by Side Comparison of the Models

Figures 4.2, 4.3 and 4.4 show a side by side comparison of the models which generate images with resolution 64×64 . Figures 4.5, 4.6 and 4.7 include a side by side comparison of the models which generate images with resolution 256×256 .

This flower is orange in color, with petals that are layered.



Figure 4.2: Each row contains 64×64 images generated by a different model from the top text description. The order is: GAN-CLS (first row), WGAN-CLS (second row), StackGAN Stage I (third row), Conditional Least Squares PGGAN (forth row), Conditional Wasserstein PGGAN (fifth row).

The lack of diversity of the images produced by GAN-CLS is evident. All the other models create a variety of images for the same text descriptions thanks to the condition

This flower is white and yellow in colour, with petals that are multi coloured.



Figure 4.3: Each row contains 64×64 images generated by a different model from the top text description. The order is: GAN-CLS (first row), WGAN-CLS (second row), StackGAN Stage I (third row), CLSPGGAN (fourth row), CWPGGAN (fifth row).

augmentation module they are all equipped with. The quality of the images generated by WGAN-CLS is subjectively better than the one of GAN-CLS and comparable to the one of Stage I of StackGAN. The CPGGANs (64×64) generate more structurally coherent images than the other models. The Wasserstein based CPGGAN generates even more diverse images, but the text-image matching of its images is slightly worse than the one of the other models. Figure 4.4, where CWPGGAN generates a few flowers which do not contain any shade of pink is one such example.

The slightly worse text-image matching becomes more visible on the 256×256 version of CWPGGAN (see 4.7). Nevertheless, the images are subjectively better than the images of the other models, which is also confirmed by the Inception Score. Note that, in the case of 256×256 images, the CLSPGGAN generates slightly unrealistic textures (Figure 4.5) or images which lack local coherence (Figure 4.7). I believe this is due to the Gaussian noise hack which was used to fix its instability.

To test that the models do not simply memorise the images from the dataset and that they produce new images, a nearest neighbour analysis is given in Figure 4.8 for 64×64 images and Figure 4.9 for 256×256 images.

A comparison between StackGAN and Conditional Wasserstein PGGAN is provided in Appendix D.

A flower with thin pink petals resting around a cluster of white and yellow stamen.



Figure 4.4: Each row contains 64×64 images generated by a different model from the top text description. The order is: GAN-CLS (first row), WGAN-CLS (second row), StackGAN Stage I (third row), Conditional Least Squares PGGAN (forth row), Conditional Wasserstein PGGAN (fifth row).

This flower is orange in color, with petals that are layered.



Figure 4.5: Each row contains 256×256 images generated by a model from the top text description. The order is: StackGAN Stage II (first row), Conditional Least Squares PGGAN (second row), Conditional Wasserstein PGGAN (third row).

This flower is white and yellow in colour, with petals that are multi coloured.



Figure 4.6: Each row contains 256×256 images generated by a model from the top text description. The order is: StackGAN Stage II (first row), Conditional Least Squares PGGAN (second row), Conditional Wasserstein PGGAN (third row).

A flower with thin pink petals resting around a cluster of white and yellow stamen.

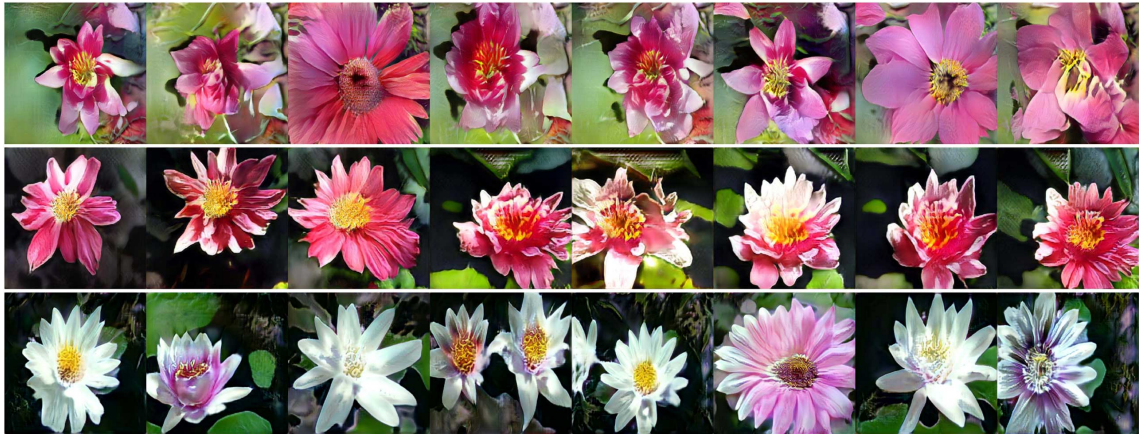
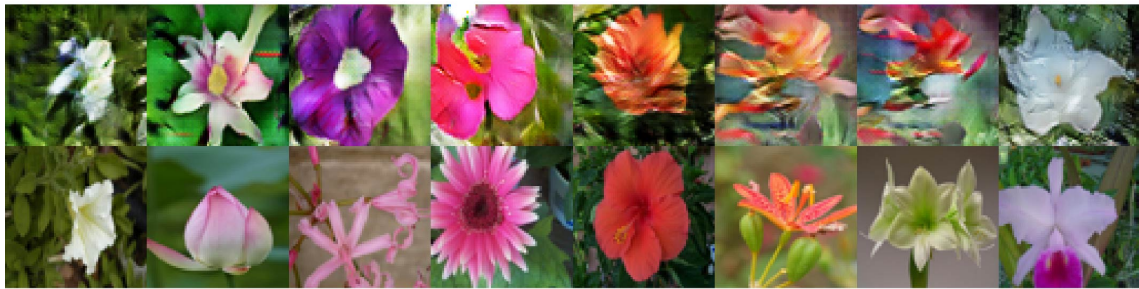
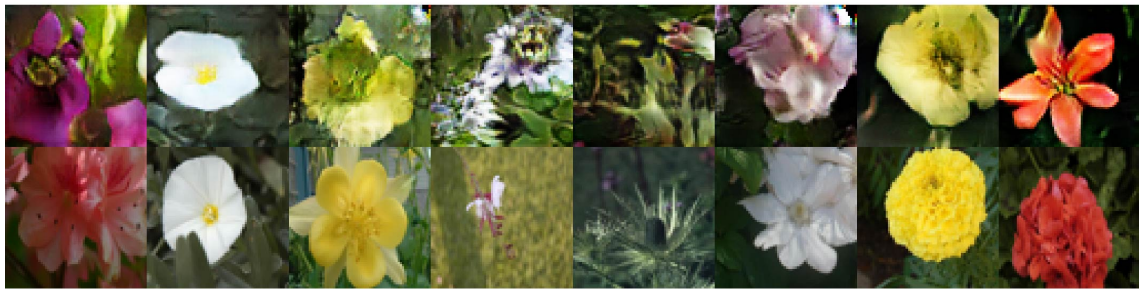


Figure 4.7: Each row contains 256×256 images generated by a model from the top text description. The order is: StackGAN Stage II (first row), Conditional Least Squares PGGAN (second row), Conditional Wasserstein PGGAN (third row).

GAN-CLS



WGAN-CLS



StackGAN Stage 1

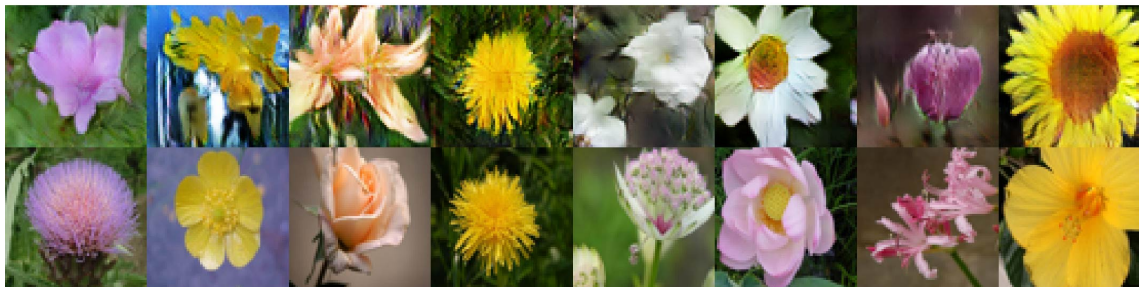


Figure 4.8: For each model, the first row contains 64×64 images produced by the model and the second row contains the nearest neighbour from the training dataset.

StackGAN Stage II



WPGGAN



Figure 4.9: For each model, the first row contains 256×256 images produced by the model and the second row contains the nearest neighbour from the training dataset.

Chapter 5

Reflection and Conclusion

This chapter contains a reflection on the planning and management of this project and ends with a conclusion of the present work.

5.1 Planning and Management

The plan of my project is divided into two main parts:

1. The first part, covering the first semester, was concerned with background reading, the understanding of the existent state of the art models, the reproduction of their results and the identification of their limitations and consequently of possible directions of research.
2. The second part, covering the second semester, was concerned with finding solutions for the identified research problems.

Milestone	Planned Weeks	Actual Weeks
Background reading on GANs	1-4	1-4
Reproduce GAN-CLS results [29]	5-6	5-7
Reproduce StackGAN results [37]	7-10	7-11
Reproduce I2T2I results [6]	11-13	-
Implement Inception Score evaluation	14	12

Table 5.1: The milestones of the first part of the project. The plan is based on the weeks of the academic year.

Table 5.1 includes the milestones for the first part of the project together with their timeline. Out of these milestones, I decided to skip the reproduction of the results of the I2T2I paper for two reasons. The StackGAN paper proposes a more elegant solution for textual data augmentation as discussed in Section 2.3.1. The implementation of an image captioning system would have required significantly more background reading in the area of language models which is vast. Instead, I decided to start the research part of the project earlier, before the start of the second semester. After the first semester, I identified three research directions summarised in Table 5.2.

Research direction	Planned Weeks	Actual Weeks
Stable Conditional GAN	15-20	13-21
Conditional GAN operating on multiple resolutions	21-25	21-26
Explicit evaluation of text-image matching	26	26

Table 5.2: The identified research directions for the second semester. The plan is based on weeks of the academic year.

Out of these research directions, I obtained good results for the first two points on the list as described in Chapter 3.

Due to the significant training time the presented models take and the limited computing resources (one Nvidia 1080Ti) I decided to focus my experiments on the flowers dataset and not on both the flowers and birds datasets as I originally intended. The focus on a slightly smaller and less complicated dataset such as Oxford-102 offered more time for testing ideas and rigorous evaluation. Nevertheless, I run a few experiments on the birds dataset, and the results can be found in Appendix D.

5.2 Conclusion

In this work, I present Generative Adversarial Networks and their application in the problem of text to image synthesis. I explain how the current state of the art models work at the intersection between Computer Vision and Natural Language Models and I reproduce the results of the papers which introduce them. Moreover, I bring my contribution to the field by proposing a novel Conditional Wasserstein GAN (WGAN-CLS) which enables conditional generation of images with a stable training procedure. The images this model generates are comparable to the current state of the art models. I show how this conditional Wasserstein loss function can be used in a more advanced model: the proposed Conditional Progressive Growing GAN. Other classical GAN loss functions would not work on such a model because of their instability during training. I show that Conditional Progressive Growing GANs, with the novel conditional Wasserstein loss, produce better results than the current state of the art models which use only sentence-level visual semantics.

Appendix A

Proof of Theorem 1.3.1

Theorem 1.3.1. *The Nash equilibrium of the (non-parametric) GAN game occurs when:*

1. *The strategy of the discriminator is $D = \frac{\mathbb{P}_r}{\mathbb{P}_r + \mathbb{P}_g}$*
2. *The strategy G of the generator makes $\mathbb{P}_g = \mathbb{P}_r$.*

Proof. First, we are going to find in function space the optimal strategy of the discriminator $D^* : \mathcal{X} \rightarrow [0, 1]$ for a fixed strategy $G : \mathcal{Z} \rightarrow \mathcal{X}$ of the generator. For this, we assume that $\mathbb{P}_r(\mathbf{x})$ and $\mathbb{P}_g(\mathbf{x})$ are non zero everywhere in order to avoid undefined behaviour. The proof uses variational calculus for finding function D which maximises the functional $F[x, D] = V(D, G)$. For an introduction to calculus of variations please consult [8].

$$\begin{aligned}
V(D, G) &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r}[\log(D(\mathbf{x}))] + \mathbb{E}_{\mathbf{Z} \sim \mathbb{P}_Z}[\log(1 - D(G(\mathbf{z})))] \\
&= \int_{\mathcal{X}} \mathbb{P}_r(\mathbf{x}) \log(D(\mathbf{x})) d\mathbf{x} + \int_{\mathcal{Z}} \mathbb{P}_Z(\mathbf{z}) (\log(1 - D(G(\mathbf{z})))) d\mathbf{z} \\
&\quad \text{(by the definition of expectation)} \\
&= \int_{\mathcal{X}} \mathbb{P}_r(\mathbf{x}) \log(D(\mathbf{x})) d\mathbf{x} + \int_{\mathcal{X}} \mathbb{P}_g(\mathbf{x}) (\log(1 - D(\mathbf{x}))) d\mathbf{x} \\
&\quad \text{(by the Radon–Nikodym theorem of measure theory)} \\
&= \int_{\mathcal{X}} [\mathbb{P}_r(\mathbf{x}) \log(D(\mathbf{x})) + \mathbb{P}_g(\mathbf{x}) (\log(1 - D(\mathbf{x})))] d\mathbf{x} \\
&\quad \text{(by the additive property of integration)} \\
&= \int_{\mathcal{X}} [\mathbb{P}_r(\mathbf{x}) \log(D(\mathbf{x})) + \mathbb{P}_g(\mathbf{x}) (\log(1 - D(\mathbf{x})))] d\mathbf{x} \\
&= \int_{\mathcal{X}} L(\mathbf{x}, D)
\end{aligned}$$

We denote here the integrand by $L(x, D)$. To find $D(\mathbf{x})$ which maximises this functional, we apply the Euler-Lagrange equation from variational calculus, which can be further simplified because D' does not show up in L . For the next part, we will not include the argument x of the functions $\mathbb{P}(\mathbf{x})$ and write them as $\mathbb{P}$ to save space.

$$\begin{aligned}
\frac{\partial L}{\partial D} - \frac{d}{dx} \frac{\partial L}{\partial D'} = 0 &\Leftrightarrow \frac{\partial L}{\partial D} - 0 = 0 \\
&\Leftrightarrow \frac{\partial}{\partial D} [\mathbb{P}_r \log(D) + \mathbb{P}_g (\log(1-D))] = 0 \\
&\Leftrightarrow \frac{\mathbb{P}_r}{D} - \frac{\mathbb{P}_g}{1-D} = 0 \\
&\Leftrightarrow \mathbb{P}_r(1-D) = \mathbb{P}_g D \\
&\Leftrightarrow \mathbb{P}_r = (\mathbb{P}_g + \mathbb{P}_r) D \\
&\Leftrightarrow D = \frac{\mathbb{P}_r}{\mathbb{P}_g + \mathbb{P}_r}
\end{aligned}$$

Now we can write the cost function of G as $C(G) = V(D^*, G)$.

$$C(G) = \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} \log\left(\frac{\mathbb{P}_r}{\mathbb{P}_r + \mathbb{P}_g}\right) + \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} \log\left(\frac{\mathbb{P}_g}{\mathbb{P}_r + \mathbb{P}_g}\right) \quad (\text{A.1})$$

We now show that the global minimum of $C(G)$ is obtained when $\mathbb{P}_g = \mathbb{P}_r$. When this equation holds, $D = \frac{1}{2}$. Plugging this in $C(G)$ we obtain:

$$\begin{aligned}
C(G) &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} \log\left(\frac{1}{2}\right) + \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} \log\left(\frac{1}{2}\right) \\
&= -\mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} \log(2) - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} \log(2) = -\log(4)
\end{aligned} \quad (\text{A.2})$$

By adding equations A.1 and A.2 we see that $-\log(4)$ is indeed the minimum value of $C(G)$

$$\begin{aligned}
C(G) - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} \log(2) - \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} \log(2) &= \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_r} \log\left(\frac{\mathbb{P}_r}{\mathbb{P}_r + \mathbb{P}_g}\right) \\
&\quad + \mathbb{E}_{\mathbf{X} \sim \mathbb{P}_g} \log\left(\frac{\mathbb{P}_g}{\mathbb{P}_r + \mathbb{P}_g}\right) \\
&\quad - \log(4)
\end{aligned}$$

By leaving only $C(G)$ on the left hand side we obtain:

$$\begin{aligned}
C(G) &= -\log(4) + KL(\mathbb{P}_r \parallel \frac{\mathbb{P}_r + \mathbb{P}_g}{2}) + KL(\mathbb{P}_g \parallel \frac{\mathbb{P}_r + \mathbb{P}_g}{2}) \\
&= -\log(4) + JS(\mathbb{P}_r \parallel \mathbb{P}_g)
\end{aligned}$$

Where KL and JS are the Kullback-Leibler divergence and the Jensen-Shannon divergence respectively. Because the Jensen-Shannon divergence is a metric it has the non-negativity and identity of indiscernibles properties. Thus, its minimum of 0 is realised only when $\mathbb{P}_r = \mathbb{P}_g$. □

Note that as in maximum likelihood models, no knowledge of the unknown $\mathbb{P}_r$ is needed to optimise $JS(\mathbb{P}_r \parallel \mathbb{P}_g)$ (only samples). The expectation over $\mathbf{X} \sim \mathbb{P}_r$ can be approximated using a sample mean.

Appendix B

Notions of Deep Learning

The goal of this chapter is to explain some of the Deep Learning notions and terminology used throughout this report. On the one hand, these notions are very commonly used and cannot be avoided. On the other hand, describing them in the body of the report would distract the reader from the main ideas.

B.1 Neural Networks

Neural Networks can be viewed as universal parametric function approximators. A function $f : A \rightarrow B$ can be approximated by a parametric function $f_{\theta} : A \rightarrow B$ where θ are the parameters of the network. A key idea in deep learning is that learning complicated functions can be done by using the composition of multiple but simple non linear functions. The stack of layers of a network is a composition of such functions. Assuming f_{θ} has n layers, these can be denoted by $f_{\theta}^{(1)}, \dots, f_{\theta}^{(n)}$. Then, $f_{\theta} = f_{\theta}^{(n)}(\dots(f_{\theta}^{(1)}))$

B.1.1 Backpropagation

In order to approximate f , a loss function L which describes how far the approximation f_{θ} is from f is used. The approximation can be improved by decreasing L . The minimisation of L offers a way to adjust the parameters θ to improve the approximation. This process is called back-propagation. For any parameter θ_i of the network, the partial derivative $\frac{\partial L}{\partial \theta_i}$ can be computed using the (multivariate) chain rule. For this partial derivative to exist, it is required that the functions each layer implements are differentiable (almost everywhere). Thus, the parameters can be updated using the following procedure: $\theta_i \leftarrow \theta_i - \alpha \frac{\partial L}{\partial \theta_i}$ where α is the learning rate. The minus sign is introduced because the parameter must be moved in the opposite direction of the sign of the derivative to approach the minimum of L .

Most often, mini-batch gradient descent is used. The network does not take as input a single example, but rather a batch of samples and the loss is computed with respect to this batch. When the parameter update is performed, it is calculated using the average derivative of that parameter where the average is taken over all the examples in the mini-batch. Thus, bigger mini-batches help reduce the variance of the updates but introduce additional computational cost at the same time.

B.1.2 Activation Functions

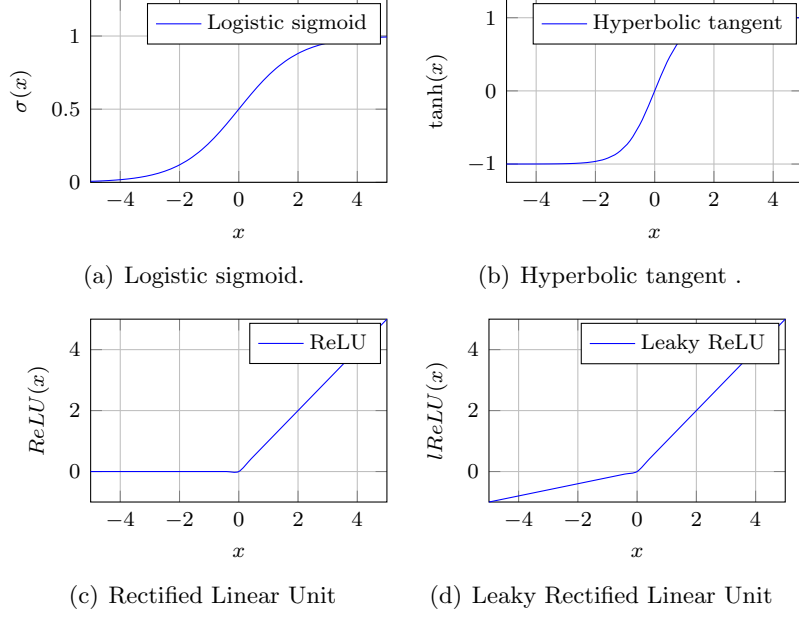


Figure B.1: The commonly used activation functions. Softmax is not depicted here because it is a multivariate vector valued function and it is harder to visualise.

Usually, $f_{\theta}^{(i)} = h(g_{\theta}^{(i)})$ where $g_{\theta}^{(i)}$ is a simple (linear) function followed by a non linearity $h(x)$. $h(x)$ is called an activation function. Without the activation functions, the network would not gain any additional capacity because the composition of multiple linear functions is still a linear function. In other words, multiple linear layers stacked together have the same capacity as a network with a single linear layer.

The activation functions (Figure B.1) commonly used in practice and in this report are:

- The sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$. It is also called the logistic function. It can be used to bring a variable in the range $[0, 1]$.
- The hyperbolic tangent activation function $\tanh(x) = 2\sigma(2x) - 1$ is a re-scaled sigmoid which brings the values in the range $[-1, 1]$. It is generally used when generating images to bring the values of the pixels in the range $[-1, 1]$.
- The softmax function takes as input a vector $\mathbf{x}$ and outputs a vector $\xi(\mathbf{x})$ whose values are in range $[0, 1]$ and sum up to one. It is usually used in classifiers to obtain a valid probability distribution over the classes some input could belong to. It is defined as $\xi(\mathbf{x})_j = \frac{x_j}{\sum_k x_k}$.
- A very popular and simple activation is the Rectified Linear Unit: $ReLU(x) = \max(0, x)$ [13]. It is used in the intermediate layers to introduce non linearity in the model. Because for $x > 0$ the derivative is constant and non-zero, this activation prevents the gradient from saturating.

- A generalisation of ReLUs are Leaky ReLUs $lReLU(x) = \max(kx, x)$ where $k \in [0, 1]$ is usually close to 0. $k = 0.2$ is a common value.

B.2 Normalisation Techniques

B.2.1 Batch Normalisation

Batch Normalisation [16] is a widely used technique for speeding up the learning and improving the performance of deep learning models. Ideally, it is desired that the input to a model to be whitened, to have zero mean and unit variance. Whitening the data was shown decades ago [21] to improve the speed of the training. Nevertheless, for deep learning, it is not enough because between layers inputs which are not normalised appear. A layer could supply to the next layer inputs with high variance and a mean far from zero. This phenomenon is called internal covariance shift. The fix is to whiten the data given as input to every layer using the batch statistics as in Equation B.1.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \text{ and } y_i = \gamma \hat{x}_i + \beta \quad (\text{B.1})$$

Here, x_i is an activation for the i^{th} example in the minibatch and $\hat{x}_i$ is its whitened version. μ and σ^2 are the mean and variance of that activation over the entire batch. The trainable parameters γ and β , ensure that this transformation is also able to represent the identity transform. They act as denormalisation parameters and can reverse the whitening if needed.

Batch normalisation is based on the assumption that the batch statistics approximate the dataset statistics. Thus, the disadvantage of batch normalisation is that for small batch sizes the approximation is not so good and the performance drops. For more details, please check [16].

GANs are empirically known to be more stable on architectures which use batch normalisation in the generator and the discriminator.

B.2.2 Layer Normalisation

Layer normalisation performs the same type of whitening as batch normalisation with the exception that the normalisation is performed over all the hidden units in a layer and not by using the mini-batch statistics.

Because this normalisation technique is independent of the size of the mini-batch, it has the advantage that it does not impose a lower bound on the batch size. Nevertheless, layer normalisation brings only marginal improvements in convolutional layers and is better suited for Recurrent Neural Networks and fully connected layers.

B.3 Convolutional Layers

Convolutional Layers are the building block of Convolutional Neural Networks. They operate on third order tensors, or informally, a 3D array of values and produce as output another 3D array (not necessarily with the same dimensions). Images are one such tensor with dimensions $\text{width} \times \text{height} \times 3$ in the case of RGB images. Convolutional layers are composed of a number f of filters which can be adjusted. A filter has a reduced spatial resolution such as 3×3 , and its depth is always equal to the depth of the input tensors. Each filter is convoluted with regions of the input tensor by sliding it across the width and height of the tensor. The result of each such convolution operation is a scalar. After sliding the filter over the spatial dimensions of the tensor, the output scalars form together a matrix. Each of the f filters produces one such matrix. All these matrices stacked together form the output tensor. Convolutional layers also have other parameters besides the number of filters and the size of the filter. One of them is the stride which determines by how many units the filter is moved in each direction during sliding. Another one is the amount of zero padding which refers to padding the borders of the input tensor with zeros. These four hyper-parameters: the number of filters, the filter size, the stride and the amount of zero padding are used to manipulate the exact shape of the output tensor (Figure B.2).

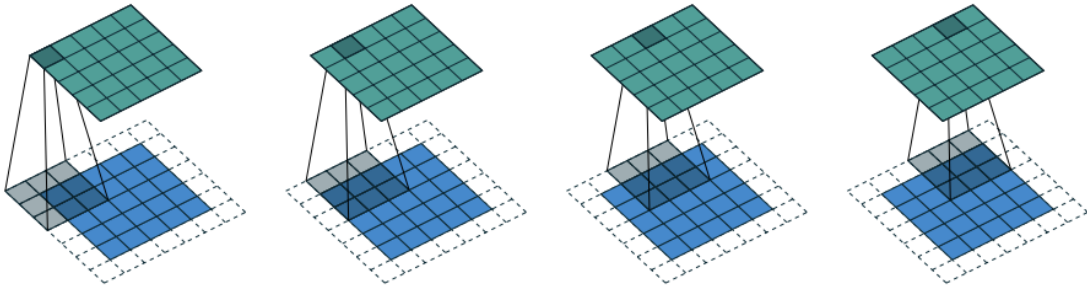


Figure B.2: A convolutional filter with padding, stride one and filter size of 3×3 . Image is taken from [7].

Each of the filters of a convolutional layer tries to learn a useful visual feature such various types of edges. The filters from the deeper levels of the network recognise more complex structures from the input image.

Convolutional layers can reduce or maintain the spatial resolution of the input tensor. Nevertheless, sometimes a reverse operation is needed to perform upsampling. A deconvolutional layer does precisely that. Deconvolutional layers can be thought of (and inefficiently implemented) as regular convolutional layers with the exception that the input pixels are moved apart, and zeros are inserted between them (see Figure B.3).

For more details on convolutional layers and their arithmetic, please consult [7].

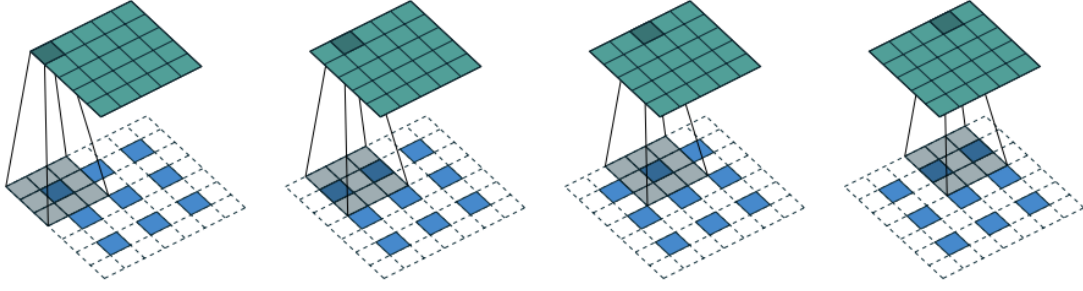


Figure B.3: The transposed convolution operation (also called deconvolution) performing upsampling. The resolution of the output (5x5) is higher than the one of the input (3x3). Image is taken from [7].

B.4 Residual Layers

A good rule of thumb in deep learning is that more layers do not always translate to better performance. In fact, only increasing the depth of deep learning models has been shown to cause a decrease in the performance of the network [12].

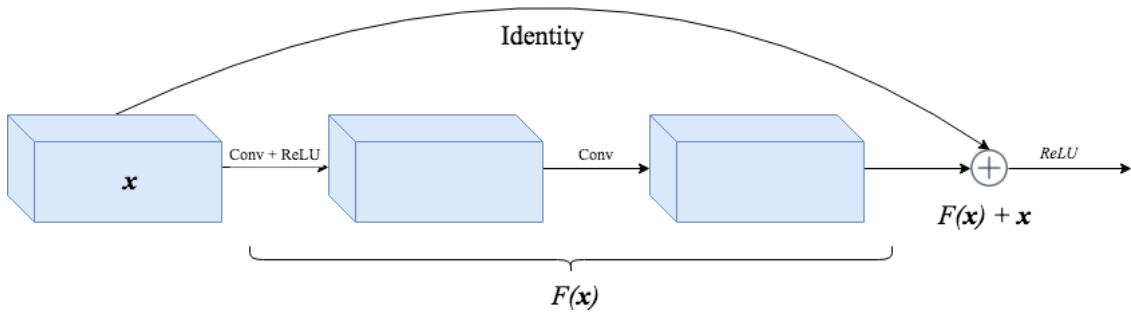


Figure B.4: A residual layer with two intermediate convolutional layers. The curved arrow represents the identity skip connection. The output of the two layers $F(\mathbf{x})$ and the input $\mathbf{x}$ are added at the end. An activation functions is applied after the addition.

Residual Layers [12] have eliminated this problem and led to better results. Figure B.4 shows the architecture of a residual layer. In this architecture, the network has to learn a function F with respect to the identity mapping, rather than the zero mapping. This approach has two advantages:

1. If the identity mapping is needed, the network can easily represent it by setting the value of the two intermediate weight layers to zero.
2. The shortest path from any layer to the output layer is shorter than in a normal network. The identity skip connection helps prevent the gradient from becoming almost zero, especially for the first layers which are very far from the output.

For more details please check [12, 14, 33].

Appendix C

Additional Explanations of Wasserstein GANs

I include here a few clarifications and intuitive explanations for the technical terms used in Section 3.1.

C.1 Manifolds

I will not formally introduce manifolds here as it would be hard to present them rigorously in such a limited space. They are in essence generalisations in n dimensions of surfaces. They are spaces with the property that the neighbourhood of any point resembles a Euclidean space. A typical example is that of Earth (the surface of a sphere) which locally (on Earth) looks flat, but the global structure is different. Low dimensional manifolds can be described by fewer dimensions than the space they live in. For example, a sheet of paper can be defined by two coordinates even if it is stretched in strange ways and it is perceived as part of the 3D world. It fundamentally remains a 2D object.

C.2 Lipschitz Functions

Lipschitz functions are functions whose rate of growth is limited. Formally, given two metric spaces $(\mathcal{A}, d_{\mathcal{A}})$ and $(\mathcal{B}, d_{\mathcal{B}})$, a function $f : \mathcal{A} \rightarrow \mathcal{B}$ is K -Lipschitz continuous for some constant K if for any $x_1, x_2 \in \mathcal{A}$, relation C.1 holds.

$$d_{\mathcal{B}}(f(x_1), f(x_2)) \leq K \cdot d_{\mathcal{A}}(x_1, x_2) \tag{C.1}$$

This work is concerned with Euclidean spaces, so $d_{\mathcal{A}}$ and $d_{\mathcal{B}}$ are the usual Euclidean distances in their respective spaces.

Appendix D

Samples Generated on the Birds Dataset

This section includes samples of generated images produced by Conditional Wasserstein PGGAN and StackGAN on the birds dataset as well as side by side comparisons between them. The birds dataset is more difficult than the flowers dataset. The dataset contains more images and more categories. The birds have different poses and are usually surrounded by vegetation which sometimes obstructs the view. Moreover, the descriptions are more detailed and refer to fine-grained details from the images. The higher difficulty of the dataset better reflects the difference between models.

Small, mostly yellow bird, with brown, white and black stripes on its wings and tail.



A bird with a small triangular bill, black cheek patch and blue plumage across its body.



Figure D.1: Samples generated by Stage I of StackGAN on the birds dataset.

The bird is white with grey wings and a yellow bill.



The bird has an orange crown and multicoloured tail feathers.

This bird has brown feathers, a white throat and an orange beak.



A small bird with a brown body and red head and throat.

Figure D.2: Samples generated by Stage I of StackGAN from interpolations of text embedding from the birds dataset.

This bird has an orange body, with black wings, tail and throat.



This pretty little bird has a fluffy white belly, a long, pointy bill, and its upper half is brown.

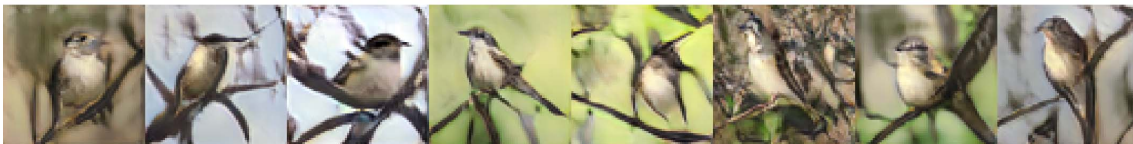


Figure D.3: Samples (64×64) generated by CWPGGAN on the birds dataset.

The bird has an orange belly and breast as well as a black bill.



This bird has a bright white body with a yellow bill, grey wings and black rectrices.

This bird has a long pointy black bill, a back that is bright blue, and outer rectrices that are bright blue.



A striking gray bird with red crown, black cheek patch and throat, and yellow, white and red wings and tail feather.

Figure D.4: Samples (64×64) generated by CWPGGAN from interpolations of text embedding from the birds dataset.

The eye is bright red, the crown of the head is grey, the throat, neck and chest are white and the body is grey.



Figure D.5: Comparison of 64×64 models on the birds dataset: Stage I of StackGAN (first row), CWPGGAN (second row).

This small gray bird has a pointed bill, black wings with white bars, and white abdomen and undertail coverts.



Figure D.6: Comparison of 64×64 models on the birds dataset: Stage I of StackGAN (first row), CWPGGAN (second row).

A greyish-blue coloured bird with a white-grey face, and big blue feet.



Figure D.7: Comparison of 64×64 models on the birds dataset: Stage I of StackGAN (first row), CWPFGAN (second row).

Small blue bird with short bill and black inner and outer rectrices.



The small bird has a blue crown and white belly.



Figure D.8: Samples (256×256) generated by Stage II of StackGAN on the birds dataset.

This light brown bird has a medium brown crest, wings, and tail.



This bird has a white belly with blue wings and a blue head.

The bird has a small beak when compared to its body, with black throat, reddish brown crown and grey belly.



This bird has a white belly and breast with black primaries and secondaries.

Figure D.9: Samples generated by Stage II of StackGAN from interpolations of text embedding from the birds dataset.

A fat round bird with a bright yellow breast and blue on top of the head.



The tan bird has a short beak, with stone black eyes.



Figure D.10: Samples (256×256) generated by CWPGGAN on the birds dataset.

The bird has a white breast and belly and a small bill.



This bird has a white with grey speckled appearance on its belly and breast, with grey crown and white supercilium.

This bird has a small yellow beak and tiny black eyes, surrounded by brown feathers, laces with dark brown stripes.



Small grey bird with a feathery crown and white wing bar and flanks.

Figure D.11: Samples (256×256) generated by CWPGGAN from interpolations of text embedding from the birds dataset.

A greyish-blue coloured bird with a white-grey face and big blue feet.



Figure D.12: Comparison of 256×256 models on the birds dataset: Stage II of StackGAN (first row), CWPGGAN (second row).

The eye is bright red, the crown of the head is grey, the throat, next and chest are white and the body is grey.



Figure D.13: Comparison of 256×256 models on the birds dataset: Stage II of StackGAN (first row), CWPGGAN (second row).

Appendix E

Conditional PGGANs

This section offers more details about the Conditional Least Squares and Wasserstein Progressive Growing GAN implementations. These details are omitted in the body of the work.

Generator	Act	Output shape	Discriminator	Act	Output shape
$z e$	-	256	Input image	-	$256 \times 256 \times 3$
Projection	linear	$4 \times 4 \times 512$	Conv 1×1	ReLU	$256 \times 256 \times 32$
Conv 3×3	ReLU	$4 \times 4 \times 512$	Conv 3×3	ReLU	$256 \times 256 \times 32$
Conv 3×3	ReLU	$4 \times 4 \times 512$	Conv 3×3	ReLU	$256 \times 256 \times 64$
Upsample	-	$8 \times 8 \times 512$	Downsample	-	$128 \times 128 \times 64$
Conv 3×3	ReLU	$8 \times 8 \times 512$	Conv 3×3	ReLU	$128 \times 128 \times 64$
Conv 3×3	ReLU	$8 \times 8 \times 512$	Conv 3×3	ReLU	$128 \times 128 \times 128$
Upsample	-	$16 \times 16 \times 512$	Downsample	-	$64 \times 64 \times 128$
Conv 3×3	ReLU	$16 \times 16 \times 512$	Conv 3×3	ReLU	$64 \times 64 \times 128$
Conv 3×3	ReLU	$16 \times 16 \times 512$	Conv 3×3	ReLU	$64 \times 64 \times 256$
Upsample	-	$32 \times 32 \times 512$	Downsample	-	$32 \times 32 \times 256$
Conv 3×3	ReLU	$32 \times 32 \times 512$	Conv 3×3	ReLU	$64 \times 64 \times 128$
Conv 3×3	ReLU	$32 \times 32 \times 512$	Conv 3×3	ReLU	$64 \times 64 \times 256$
Upsample	-	$64 \times 64 \times 512$	Downsample	-	$32 \times 32 \times 256$
Conv 3×3	ReLU	$64 \times 64 \times 256$	Conv 3×3	ReLU	$32 \times 32 \times 256$
Conv 3×3	ReLU	$64 \times 64 \times 256$	Conv 3×3	ReLU	$32 \times 32 \times 512$
Upsample	-	$128 \times 128 \times 256$	Downsample	-	$16 \times 16 \times 512$
Conv 3×3	ReLU	$128 \times 128 \times 128$	Conv 3×3	ReLU	$16 \times 16 \times 512$
Conv 3×3	ReLU	$128 \times 128 \times 128$	Downsample	-	$8 \times 8 \times 512$
Upsample	-	$256 \times 256 \times 128$	Conv 3×3	ReLU	$8 \times 8 \times 512$
Conv 3×3	ReLU	$256 \times 256 \times 64$	Conv 3×3	ReLU	$8 \times 8 \times 512$
Conv 3×3	ReLU	$256 \times 256 \times 64$	Downsample	-	$4 \times 4 \times 512$
Conv 2×2	ReLU	$256 \times 256 \times 9$	Embedding concatenation	ReLU	$4 \times 4 \times 640$
Conv 1×1	linear	$256 \times 256 \times 3$	Conv 3×3	ReLU	$4 \times 4 \times 512$
			Conv 4×4	ReLU	$1 \times 1 \times 512$
			Fully connected	linear	$1 \times 1 \times 1$

Table E.1: All the layers of the Conditional PGGAN generator (left) and discriminator (right). All the stages are separated by horizontal lines.

E.1 More on Architecture and Training

To be able to fit the model in memory at the high-resolution stages, small batch sizes have to be used. I use a batch size of 16 until the 64×64 resolution is reached (including this resolution). For the higher resolutions, I use a batch size of 8. Because the performance of batch normalisation depends on bigger batch sizes, I use layer normalisation in the generator. In the discriminator, I use layer normalisation only for CLSPGGAN. The detailed architecture of the Conditional PGGAN is given in Table E.1.

For training the Conditional Wasserstein PGGAN I use $n_{critic} = 1$ and a learning rate of 0.0001 both for the generator and the discriminator. The other parameters are $\alpha = 1$, $\lambda = 150$, $\rho = 8$. The Adam optimiser is used with $\beta_1 = 0$ and $\beta_2 = 0.99$.

For training the Conditional Least Squares PGGAN I use a learning rate of 0.0001 both for the generator and the discriminator and $\rho = 8$. The Adam optimiser is used with $\beta_1 = 0.5$ and $\beta_2 = 0.9$.

Bibliography

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] Martín Arjovsky and Léon Bottou. Towards principled methods for training generative adversarial networks. *CoRR*, abs/1701.04862, 2017.
- [3] Martín Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR, 2017.
- [4] Jimmy Ba, Volodymyr Mnih, and Koray Kavukcuoglu. Multiple object recognition with visual attention. *CoRR*, abs/1412.7755, 2014.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.
- [6] Hao Dong, Jingqing Zhang, Douglas McIlwraith, and Yike Guo. I2t2i: Learning text to image synthesis with textual data augmentation. *2017 IEEE International Conference on Image Processing (ICIP)*, pages 2015–2019, 2017.
- [7] Vincent Dumoulin and Francesco Visin. A guide to convolution arithmetic for deep learning. *ArXiv e-prints*, mar 2016.
- [8] I.M. Gelfand and S.V. Fomin. *Calculus of Variations*. Dover Books on Mathematics. Dover Publications, 2012.

- [9] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 2672–2680. Curran Associates, Inc., 2014.
- [10] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pages 5769–5779. Curran Associates, Inc., December 2017. arxiv: 1704.00028.
- [11] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, June 2016.
- [12] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, June 2016.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *CoRR*, abs/1502.01852, 2015.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *ECCV*, 2016.
- [15] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 6626–6637. Curran Associates, Inc., 2017.
- [16] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32Nd International Conference on International Conference on Machine Learning - Volume 37, ICML’15*, pages 448–456. JMLR.org, 2015.
- [17] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *CoRR*, abs/1710.10196, 2017.
- [18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014.
- [19] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *CoRR*, abs/1312.6114, 2013.

- [20] Ryan Kiros, Yukun Zhu, Ruslan Salakhutdinov, Richard S. Zemel, Antonio Torralba, Raquel Urtasun, and Sanja Fidler. Skip-thought vectors. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'15, pages 3294–3302, Cambridge, MA, USA, 2015. MIT Press.
- [21] Yann LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade, This Book is an Outgrowth of a 1996 NIPS Workshop*, pages 9–50, London, UK, UK, 1998. Springer-Verlag.
- [22] Xudong Mao, Qing Li, Haoran Xie, Raymond Y.K. Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks. *arXiv preprint arXiv:1611.04076*, 2016.
- [23] Volodymyr Mnih, Nicolas Heess, Alex Graves, and Koray Kavukcuoglu. Recurrent models of visual attention. *CoRR*, abs/1406.6247, 2014.
- [24] Hariharan Narayanan and Sanjoy Mitter. Sample complexity of testing the manifold hypothesis. In J. D. Lafferty, C. K. I. Williams, J. Shawe-Taylor, R. S. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems 23*, pages 1786–1794. Curran Associates, Inc., 2010.
- [25] M-E. Nilsback and A. Zisserman. Automated flower classification over a large number of classes. In *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*, Dec 2008.
- [26] Henning Petzka, Asja Fischer, and Denis Lukovnikov. On the regularization of wasserstein GANs. In *International Conference on Learning Representations*, 2018.
- [27] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *CoRR*, abs/1511.06434, 2015.
- [28] Scott Reed, Zeynep Akata, Bernt Schiele, and Honglak Lee. Learning deep representations of fine-grained visual descriptions. In *IEEE Computer Vision and Pattern Recognition*, 2016.
- [29] Scott Reed, Zeynep Akata, Xinchun Yan, Lajanugen Logeswaran, Bernt Schiele, and Honglak Lee. Generative adversarial text-to-image synthesis. In *Proceedings of The 33rd International Conference on Machine Learning*, 2016.
- [30] Tim Salimans, Ian J. Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *NIPS*, 2016.
- [31] C. Szegedy, Wei Liu, Yangqing Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, June 2015.

- [32] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016.
- [33] Andreas Veit, Michael J. Wilber, and Serge J. Belongie. Residual networks behave like ensembles of relatively shallow networks. In *NIPS*, 2016.
- [34] Cédric Villani. *Cyclical monotonicity and Kantorovich duality*, pages 51–92. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.
- [35] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie. The Caltech-UCSD Birds-200-2011 Dataset. Technical Report CNS-TR-2011-001, California Institute of Technology, 2011.
- [36] Tao Xu, Pengchuan Zhang, Qiuyuan Huang, Han Zhang, Zhe Gan, Xiaolei Huang, and Xiaodong He. Attngan: Fine-grained text to image generation with attentional generative adversarial networks. *CoRR*, abs/1711.10485, 2017.
- [37] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, and Dimitris Metaxas. Stackgan: Text to photo-realistic image synthesis with stacked generative adversarial networks. In *ICCV*, 2017.
- [38] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, and Dimitris N. Metaxas. Stackgan++: Realistic image synthesis with stacked generative adversarial networks. *CoRR*, abs/1710.10916, 2017.